MAXUpdate

Revision 5.6.10

©2026 ECi Macola/MAX, LLC.
5455 Rings Rd
Suite 400
Dublin, OH 43017
Phone: 1-614-410-2600

All rights reserved. No part of this work may be reproduced, copied, adapted, or transmitted in any form or by any means without written permission from ECI.

Trademarks

MAX is a trademark or registered trademark of ECI Macola/MAX, LLC.

Product names mentioned in this document may be trademarks or registered trademarks of their respective companies and are hereby acknowledged. All product names that are known to be trademarks or service marks have been appropriately capitalized.

Use of a name in this document is for identification purposes only and should not be regarded as affecting the validity of any trademark or service mark, or as suggesting any affiliation between ECI and the trademark proprietor.

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THE DOCUMENT. Any product and related material disclosed herein are only furnished pursuant and subject to the terms and conditions of a license agreement. The only warranties made, remedies given, and liability accepted by ECI, if any, with respect to the products described in this document are set forth in such license agreement. ECI cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, indirect, special, or consequential damages.

Contents

1. INTRODUCTION 6
   - 1.1 MAXUpdate Overview 6
   - 1.2 Scope 6
   - 1.3 Expectations 6
   - 1.4 DLL Dependency Diagram 8
   - 1.5 Guidance 9
   - 1.6 What's New in This Release 9
2. MAX DLL DESCRIPTIONS 10
3. PREREQUISITES 10
4. FUNCTION DESCRIPTIONS (XML) 11
   - 4.1 Initialization 11
   - 4.2 Parameters 12
   - 4.3 Interface to MAXORDR2.DLL 13
   - 4.4 Interface to MAXTRAN2.DLL 32
5. XML TABLE STRUCTURE 33
   - 5.1 Formatting of XML Data 33
   - 5.2 Sales Order Master XML Sample 34
   - 5.3 Purchase Order Code XML Sample 35
   - 5.4 Shop Order XML Sample 37
   - 5.5 XML Structure for Transactions 38
6. TRANSACTION RULES 42
   - 6.1 General 42
   - 6.2 Lot/Serial Tracking (XML) 42
   - 6.3 Lot/Serial Tracking (Data Structures) 42
   - 6.4 Supported Transactions 44
   - 6.5 Supported Orders 45
   - 6.6 Transaction Descriptions 46
   - 6.7 Required Fields Matrix 47
7. USAGE 84
8. VB6 PROGRAMMING WITH MAXUPDATE. 85

8.1 Introduction 85
8.2 Writing your application 86

9. C++ PROGRAMMING WITH MAXUPDATE. 87

10. PROGRAMMING ETL WITH MAXUPDATE.NET 88
10.2 Using MAXETLTst.exe the Extract\Load Sample Application 88
10.3 Writing your Extract\Load application 95
10.4 MAX Business Objects, XML Elements and Table Names 96

11. SUPPORT FOR ASP PROGRAMMING 97
11.1 Setting up the ASP.NET environment 98

12. FUNCTION DESCRIPTIONS 99
12.1 MAXORDER2 DLL 100
12.2 MAXTRAN2 DLL 121

13. DATABASE CHANGES 130
13.1 MAX 5.5.0 to MAX 5.5.1 130

15. GLOSSARY 131

16. FREQUENTLY ASKED QUESTIONS 132

17. COMMON PROBLEMS 133
17.1 Application will not compile 133
17.2 Application does not run 133
17.3 Lot / Serial Control Problems 133
17.4 Error Messages in Application 133
17.5 MAXUpdate Version Control 133
17.6 Feedback 133

18. MAXTRANS ERROR MESSAGES 134

19. WORKING XML OF TRANSACTIONS 141
19.1 Receipt - Non-Inventory 141
19.2 Receipt – Purchase 141
19.3 Receipt – RMA 141
19.4 Receipt – Shop 142
19.5 Receipt – Subcontract 142
19.6 Receipt – Unplanned 142
19.7 Issue – Shop 143

19.8 Issue – Subcontract 143
19.9 Issue – Unplanned 143
19.10 Shipment - Sales Order 144
19.11 Shipment - Subcontract BOM 144
19.12 Transfer 144
19.13 Adjustment 145
19.14 Cycle-Count 145
19.15 Repetitive 145
19.16 Receipt – Backflush 146
19.17 Labor - Login Indirect 146
19.18 Labor - Login Order 146
19.19 Labor - Logout Indirect 147
19.20 Labor - Logout Order 147
19.21 AddSalesOrderHeader 148
19.22 ChangeSalesOrderHeader 149
19.23 DeleteSalesOrderXML(ordernum) 149
19.24 AddSalesOrderLine 150
19.25 ChangeSalesOrderLine 151
19.26 DeleteSalesOrderLineItemXML(ordernum) 152
19.27 ConvertQuoteXML(ordernum, ref bSuppressMesaage) 152
19.28 AddRMA 153
19.29 ChangeRMA 154
19.30 DeleteRMAXML(RMANUM, RMALIN, RMADEL) 154

1. Introduction

1.1 MAXUpdate Overview

MAXUpdate is a package that includes a programmer's reference and sample code for the MAX Manufacturing Software package. This Application Programmer's Interface (API) package should allow an experienced programmer to build custom interfaces that interact with MAX function code without having to worry about how to update the MAX database.

MAXUpdate was developed to extend the foundation built by the MAX for Windows modules. At present, MAXUpdate supports Sales Order, Purchase Order and Shop Order Entry as well as Shipping, Purchasing, Inventory and Shop Floor Control transaction functions.

MAXUpdate provides the experienced programmer with tools and an example method with which to program applications that "talk to" the MAX database. Generally professional work experience in .NET software development is recommended before considering working at this level. Without prior experience, the learning curve is steep. It is highly recommended to have experience in both programming and database management in order to use MAXUpdate.

1.2 Scope

The integration components included within MAXUpdate are not a sufficient tool set to integrate a CRM application with MAX. MAXUpdate describes how to call a specific set of order and transaction functions that obey MAX business rules to update the MAX database.

MAXUpdate is used ideally in situations where the programmer needs the database functionality of MAX, but would like to craft custom user interfaces using the development platform of their choosing. It would be suitable under platforms such as .NET, Visual Basic, C++, etc.

Since MAXUpdate is internally maintained by ECI, the application programmer is relieved from the chore of updating the Data Base routines whenever there is a logic or DB field change. This allows the programmer to focus on the user interface (screen) aspect of the application. On the other hand, the programmer using MAXUpdate does not have access to the details of the code contained within the DLLs. This means that custom logic changes for a particular application cannot be accommodated.

1.3 Expectations

When making function calls to MAXUpdate **all** fields need to be initialized and passed valid data. This is a tool for developers familiar with how MAX works and what data it requires. The MAXUpdate calls do not do lookups for default values for any fields (unless specifically mentioned in the descriptions) and do not fill in any default values into the fields. Some fields do have validation to make sure completely invalid data is not passed however many do not. The best way to validate what data is required by MAX would be to create a record using the MAX UI and then looking in the database to verify what fields were filled in, all fields that are not a blank string must be passed in to the MAXUpdate function call. It is possible using MAXUpdate to populate data into the database which will cause certain MAX functionality to break, it is the responsibility of the developer to ensure that their application is correctly populating all fields.

The MAX Technical Support agreement does not cover corrupt data caused by passing invalid data into a MAXUpdate function. Before calling support on an issue with any orders created or modified using MAXUpdate you should always verify the problem can be duplicated on a record that has not been affected by your application which uses MAXUpdate. If the problem cannot be duplicated please verify what values are populated in the database when using the MAX UI versus what is populated using the custom application, and make sure your application is populating all values are being passed correctly. Only contact support after verifying that complete and correct data is being passed to the MAXUpdate function and it is still populating the database differently

than the MAX UI does. If the problem cannot be duplicated within the MAX UI, and you have validated that the values populated in the database from the code match those populated by entering the data through the MAX UI, then the problem exists with the custom code and is outside the scope of the support coverage. For assistance troubleshooting your custom code, please contact the services team to assist you with the code or setup of MAXUpdate.

1.4 DLL Dependency Diagram

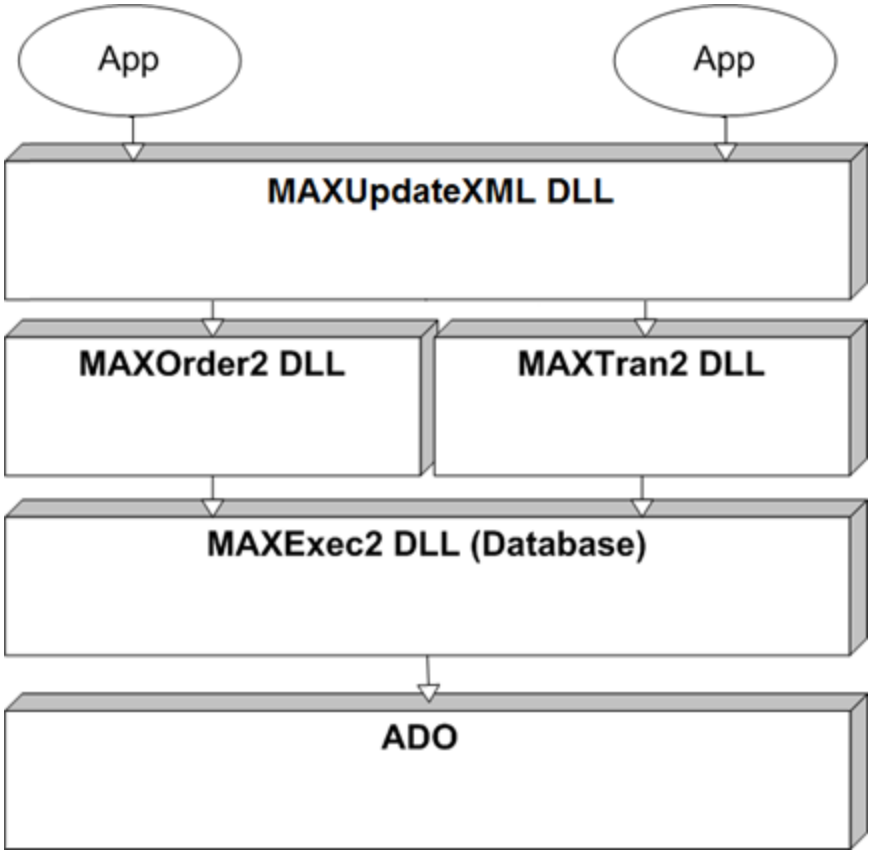
The following diagram shows the interdependencies of the Dynamic Link Libraries (DLLs) used by the MAX for Windows system.

Using DLLs has at least three key advantages over using a static library: 1) The DLL can be used by multiple "client" applications, saving memory and gaining speed, 2) All common code is centralized in the DLLs, which greatly reduces logic errors, 3) Should the DLL need to be revised, only it has to be replaced, not the entire application.

MAX modules make calls to the libraries from the top down; i.e. a module can call any module which is below it on the chart, but never vice-versa.

The MAXUpdateXML wrapper is an optional layer that can be used to simplify the data manipulation required for MAXUpdate by allowing the user to pass XML data rather than formatted data structures. Therefore, in general, when new functionality is added to MAX any applications calling the MAXUpdateXML wrapper will continue to function without any changes, where an application not using the wrapper will at a minimum need to be recompiled with the new data structure changes and re-distributed.

NOTE: When programming in a .NET language it is required to use the MAXUpdateXML wrapper, .NET does not allow defining a data structure that will work with the standard MAXUpdate DLL calls.



1.5 Guidance

Generally, custom development will require multiple development tools. The MAX support program does not cover support for these tools. The tool providers support the tools and supply a great deal of “how-to” information via the Internet. An excellent site is the Microsoft web site:

http://msdn.microsoft.com/

1.6 What’s New in This Release

ID	Module	Description	Reference
MFGR04-338	MAXUpdate	SR-MaxUpdate ChangeRequirementMAXORD int ICurDue Value	Service Cloud Case #04004490
MFGR04-366	MAXUpdate	SR- MAX Update Sales Order Header Creation & Modification	Service Cloud Case #04107460

ID	Description	Additional Note
MFGR04-338	SR-MaxUpdate ChangeRequirementMAXORD int ICurDue Value	We have addressed the issue regarding the "Current Due Date" parameter in the MaxUpdate API functions, specifically ChangeRequirementMAXORD and AddRequirementMAXORD. Users were experiencing difficulties in determining the correct format for the integer-based field, which led to errors and popups when incorrect values were used. Our team has provided additional documentation to clarify the expected format for this parameter, ensuring that users can now accurately input the required data without encountering conversion errors. This enhancement will allow for seamless updates to BOM Material Requirements within Work Orders using MaxUpdate.
MFGR04-366	SR- MAX Update Sales Order Header Creation & Modification	A defect in the MAX system, introduced in version 5.6.9, was causing errors in the creation and modification of sales order headers. This issue resulted in incorrect data entries, such as missing or erroneous values in certain fields, and problems with the 'Created By' information. These errors also affected the ability to correctly process line items and retrieve part prices. To resolve the issue, a new DLL and an updated batch installer have been provided to ensure that sales order headers are now created and modified correctly, eliminating the previous errors and ensuring smooth operation.

2. MAX DLL Descriptions

MAXUPDATEXML - The MAXUpdateXML DLL is an alternate interface to the DLLs listed below. Instead of formatted structures the functions calls in this DLL take XML strings. When programming in a .NET language the other DLLs will not function properly and this is the only interface available for MAXUpdate.

MAXORDR2 - The MAXOrdr2 DLL is used to update orders in the MAX system. This includes adding, changing and deleting line items where applicable (e.g. Sales Orders). Currently, the DLL includes functionality for Sales Orders, Forecast Orders and Purchase Orders

MAXTRAN2 - The MAX Transaction DLL handles inventory and shop floor control transactions. Examples include Cycle Count, Stock transfer, Receipt to Stock, Completions, Time Ticket operations, etc. This library is unique in that all transactions are accessed through a general-purpose function call (*See the* Function Description *for additional information*).

MAXEXEC2 - This is the low-level interface to the MAX database. It contains many support and utility routines that are used by the higher-level calls. Currently, these base routines are not documented or supported in the MAX Update package.

3. Prerequisites

Customers MUST have **MAX 5.5 or higher** installed to use all of the functions listed in the next section. Not all functions are available in all releases of the software, there have been functions added at various batches.

4. Function Descriptions (XML)

The following is a description of the various calls contained in the MAXUpdateXML Library. The functions are defined using the VB Language syntax followed by the C# Language Syntax. When using a .NET language the parameter and return detail is available in the tooltip for the function. See the section below for more detail on how to properly format the XML strings.

These functions will only process one record with each call. Do not attempt to pass multiple records in a single XML string.

4.1 Initialization

- **Sub Initialize (PriConnectionString As String, CompanyName As String, LicensePath As String, LogPath As String, bErrRpt As Boolean)**
- **void Initialize (string PriConnectionString, string CompanyName, string LicensePath, string LogPath, bool bErrRpt)**
- **Sub InitASPXMLWrapper (PriConnectionString As String, CompanyName As String, LicensePath As String, LogPath As String, bErrRpt As Boolean)**
- **bool InitASPXMLWrapper (string PriConnectionString, string CompanyName, string LicensePath, string LogPath, bool bErrRpt)**

Parameter	Description
PriConnectionString	The primary database connection string. This connection string points to the ExactMAX database.
CompanyName	The MAX company that will have data inserted or updated.
LicensePath	Path to the ExactRM.lic file.
LogPath	Path to where the log file will be written to.
ErrRpt	Flag to specify if an error report should be generated.

Returns
None.

Comments
If the PriConnectionString, CompanyName, LicensePath are left blank, the dll will use the settings stored in the registry to fill in these three parameters.

This call must be made before using any of the other MAXUpdateXML DLL calls. It does all the setup necessary to initialize the MAX DLLs and manages the handles to the objects internally.

When using MAXUpdate in an ASP environment the ASPXMLWrapperClass should be used instead of the XMLWrapperClass, also the InitASPXMLWrapper function should be used for initializing the class. This is the only difference when using MAXUpdateXML from a standard Application than an ASP Application, other than the environmental setup requirements found in a latter section.

- **Function SetVisualErrorReportingXML (ByVal ErrorReporting As Long) As Long**
- **int SetVisualErrorReportingXML (int ErrorReporting)**

Parameter	Description
ErrorReporting	Enable or disable visual error reporting (See above.)

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Use this call to enable or disable visual error reporting "on the fly".

4.2 Parameters

- **AutoCreatePrtVen As Boolean**

Default Value
False.

Comments
Determines if functions AddPurchaseRequisitionXML, ChangePurchaseRequisitionXML, AddPurchaseOrderLineItemXML and ChangePurchaseOrderLineItemXML will automatically create a Part_Vendor record if one does not exist for the Part and Vendor the order is being created for.

- **SaveBillAndRouting As Boolean**

Default Value
True.

Comments
Determines if the function AddupdateShopOrderXML will automatically create Job_Progress and Requirement_Detail records for the order.

4.3 Interface to MAXORDR2.DLL

4.3.1 Forecast Orders

- **Function AddForecastOrderXML (ByVal PartID As String, ByVal CurDue As String, ByVal CurQty As Double, ByVal OrdRef As String) As Integer**
- **short AddForecastOrderXML (string PartID, string CurDue, double CurQty, string OrdRef)**

Parameter	Description
PartID	MAX Part Number.
CurDue	Current Due Date (YYMMDD)
CurQty	Current quantity.
OrdRef	Order Reference string.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Forecast Order. Requires CurQty argument to be given in Sales unit of measure

- **Function ChangeForecastOrderXML (ByVal OrdNum As String, ByVal Status As Byte, ByVal CurDue As String, ByVal CurQty As Double, ByVal OrdRef As String) As Integer**
- **short ChangeForecastOrderXML (string OrdNum, string Status, string CurDue, double CurQty, string OrdRef)**

Parameter	Description
OrdNum	MAX Forecast Order Number
Status	Status of the Forecast Order
CurDue	Current Due Date (YYMMDD)
CurQty	Current quantity.
OrdRef	Order Reference string.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Sales Order header. Value passed in ORDNUM_27 must be a valid existing Sales Order Number.

- **Function DeleteForecastOrderXML (ByVal OrdNum As String) As Integer**
- **short DeleteForecastOrderXML (string OrdNum)**

Parameter	Description
OrdNum	MAX Forecast Order Number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Sales Order.

4.3.2 Sales Orders

- **Function AddSalesOrderXML (sXML As String) As String**
- **string AddSalesOrderXML (string sXML)**

Parameter	Description
sXML	Requires SO_Master_Table XML structure.

Returns
A String. The Sales Order number added.

Comments
Creates a new MAX Sales Order header. Pass a blank in the ORDNUM_27 field to use the next available Order Number.

- **Function AddSOXML (sXML As String, bSuppressMsg As Boolean) As String**
- **string AddSOXML (string sXML, ref bool bSuppressMsg)**

Parameter	Description
sXML	Requires SO_Master_Table XML structure.
bSuppressMsg	Pass True to suppress all message boxes associated with SO Creation

Returns
A String. The Sales Order number added.

Comments
Creates a new MAX Sales Order header. Pass a blank in the ORDNUM_27 field to use the next available Order Number. Recommended for ASP applications and other uses requiring no pop-up message boxes be displayed.

- **Function ChangeSalesOrderXML (sXML As String) As Integer**
- **short ChangeSalesOrderXML (string sXML)**

Parameter	Description
sXML	Requires SO_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an MAX Sales Order header. Value passed in ORDNUM_27 must be a valid existing Sales Order Number.

- **Function DeleteSalesOrderXML (ByVal OrdNum As String) As Integer**
- **short DeleteSalesOrderXML (string OrdNum)**

Parameter	Description
OrdNum	MAX Sales Order Number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an MAX Sales Order, including all line items and notes.

- **Function AddSalesOrderLineItemXML (sXML As String) As Integer**
- **short AddSalesOrderLineItemXML (string sXML)**

Parameter	Description
sXML	Requires SO_Detail_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Sales Order line item.

- **Function AddSOLineItemXML (sXML As String, bSuppressMsg As Boolean) As Integer**
- **short AddSOLineItemXML (string sXML, ref bool bSuppressMsg)**

Parameter	Description
sXML	Requires SO_Detail_Table XML structure.
bSuppressMsg	Pass True to suppress all message boxes associated with SO Creation

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Sales Order line item. Recommended for ASP applications and other uses requiring no pop-up message boxes be displayed.

- **Function ChangeSalesOrderLineItemXML (sXML As String, sOldSoeDetXML As String) As Integer**
- **short ChangeSalesOrderLineItemXML (string sXML, string sOldSoedetXML)**

Parameter	Description
sXML	Requires SO_Detail_Table XML structure.
sOldSoeDetXML	Requires SO_Detail_Table XML structure. Must match data existing in the database

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an MAX Sales Order header. Value passed in ORDNUM_27 must be a valid existing Sales Order Number.

- **Function DeleteSalesOrderLineItemXML (Order As String) As Integer**
- **short DeleteSalesOrderLineItemXML (string Order)**

Parameter	Description
Order	Order Number including line and delivery

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an MAX Sales Order line item, including notes.

- **Function ConvertQuoteXML (ByVal Order As String, bSuppressMsg As Boolean) As Long**
- **int ConvertQuoteXML (string Order, ref bool bSuppressMsg)**

Parameter	Description
Order	Quote number
bSuppressMsg	If set to TRUE, message boxes for errors will be suppressed.

Returns
An integer. Value is 0 on failure.

Comments
The value returned is 0 in case of an error or the number of the newly created Sales Order.

4.3.3 Sales Price

- **Function GetUnitPriceXML (ByVal Part As String, ByVal Qty As Double, ByVal CustID As String) As Double**
- **double GetUnitPriceXML (string Part, double Qty, string CustID)**

Parameter	Description
Part	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID (optional)

Returns
A double. The unit price.

Comments
This is the price with any applicable discounts.

- **Function GetStandardPriceXML (ByVal Part As String, ByVal Qty As Double, ByVal CustID As String) As Double**
- **double GetStandardPriceXML (string Part, double Qty, string CustID)**

Parameter	Description
Part	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID (optional)

Returns
A double. The standard price.

Comments
This is the price with no discounts.

- **Function GetDiscountXML (ByVal Part As String, ByVal Qty As Double, ByVal CustID As String) As Double**
- **double GetDiscountXML (string Part, double Qty, string CustID)**

Parameter	Description
Part	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID

Returns
A double. The discount percentage.

Comments
The value returned is the discount percentage, (e.g. 20 for a 20% discount).

4.3.4 RMA

- **Function AddRMAXML (sXML As String, ByRef sRMANumOut As String) As Integer**
- **short AddRMAXML (ref string sXML, ref string sRMANumOut)**

Parameter	Description
sXML	Requires RMA_Detail_Table XML structure.
sRMANumOut	Returns the RMA Number generated

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX RMA. Leave the RMANUM_52 field blank to auto-generate the RMA number.

- **Function ChangeRMAXML (sXML As String) As Integer**
- **short ChangeRMAXML (ref string sXML)**

Parameter	Description
sXML	Requires RMA_Detail_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates an existing MAX RMA.

- **Function DeleteRMAXML (ByVal RMANum As String, ByVal RMALine As String, ByVal RMADel As String) As Integer**
- **short DeleteRMAXML (string RMANum, string RMALine, string RMADel)**

Parameter	Description
RMANum	RMA Number.
RMALine	RMA Line.
RMADel	RMA Delivery.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX RMA.

4.3.5 Purchase Orders

- **Function ChangePOHeadingXML (sXML As String) As Integer**
- **short ChangePOHeadingXML (string sXML)**

Parameter	Description
sXML	Requires Purchase_Order_Code_Table XML Structure

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order header.

- **Function DeletePurchaseOrderXML (ByVal OrdNum as String) As Integer**
- **short DeletePurchaseOrderXML (string OrdNum)**

Parameter	Description
OrdNum	MAX Order number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing Exact Purchase Order heading, the associated line items and notes.

- **Function AddPurchaseOrderLineItemXML (sXML As String, ByVal IncOrdRev As Boolean, ByVal CreateHeader As Boolean) As Integer**
- **short AddPurchaseOrderLineItemXML (string sXML, bool IncOrdRev, bool CreateHeader)**

Parameter	Description
sXML	Requires Purchase_Order_Code_Table XML structure. With Order_Master record
IncOrdRev	If TRUE, increments the PO revision number
CreateHeader	If TRUE, creates a PO header record in *POCode.dat*.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Purchase Order line item. No fields in the Purchase_Order_Code need data. See example for proper nesting of the record. The value of the AutoCreatePrtVen property will determine if this function will create a Part Vendor record if one does not already exist.

- **Function ChangePurchaseOrderLineItemXML (sXML As String, ByVal IncOrdRev As Boolean) As Integer**
- **short ChangePurchaseOrderLineItemXML (string sXML, bool IncOrdRev)**

Parameter	Description
sXML	Requires Purchase_Order_Code_Table XML structure. With Order_Master record
IncOrdRev	If TRUE, increments the PO revision number

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order line item. No fields in the Purchase_Order_Code need data. See example for proper nesting of the record. The value of the AutoCreatePrtVen property will determine if this function will create a Part Vendor record if one does not already exist.

- **Function ChangePOInTransitXML (sXML As String) As Integer**
- **short ChangePOInTransitXML (string sXML)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order Line Item which is In Transit.

- **Function DeletePurchaseOrderLineItemXML (sXML As String) As Integer**
- **short DeletePurchaseOrderLineItemXML (string sXML)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Purchase Order line item.

- **Function AddNonInventoryPOLineItemXML (sXML As String, ByVal Description As String, ByVal IncOrdRev As Boolean, ByVal CreateHeader As Boolean) As Integer**
- **short AddNonInventoryPOLineItemXML (string sXML, string Description, bool IncOrdRev, bool CreateHeader)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure
Description	Description for NI PO Line Item
IncOrdRev	If TRUE, increments the PO revision number
CreateHeader	If TRUE, creates a PO header record in *POCode.dat*.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Non-Inventory Purchase Order Line Item.

- **Function ChangeNonInventoryPOLineItemXML (sXML As String, ByVal Description As String, ByVal IncOrdRev As Boolean) As Integer**
- **short ChangeNonInventoryPOLineItemXML (string sXML, string Description, bool IncOrdRev)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure
Description	Description for NI PO Line Item
IncOrdRev	If TRUE, increments the PO revision number

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Order Line Item.

- **Function ChangeNonInventoryPOInTransitXML (sXML As String) As Integer**
- **short ChangeNonInventoryPOInTransitXML (string sXML)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Order Line Item that is In Transit.

- **Function DeleteNonInventoryPOLineItemXML (sXML As String) As Integer**
- **short DeleteNonInventoryPOLineItemXML (string sXML)**

Parameter	Description
sXML	Requires Order_Master_Table XML structure

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Non-Inventory Purchase Order Line Item.

4.3.6 Purchase Requisitions

- **Function AddPurchaseRequisitionXML (sXML As String, sOrdNumOut As String) As Integer**
- **short AddPurchaseRequisitionXML (string sXML, ref string sOrdNumOut)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure
sOrdNumOut	Returns the Requisition number created

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Purchase Requisition. The value of the AutoCreatePrtVen property will determine if this function will create a Part Vendor record if one does not already exist.

- **Function ChangePurchaseRequisitionXML (sXML As String) As Integer**
- **short ChangePurchaseRequisitionXML(string sXML)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Requisition. The value of the AutoCreatePrtVen property will determine if this function will create a Part Vendor record if one does not already exist.

- **Function DeletePurchaseRequisitionXML (sXML As String) As Integer**
- **short DeletePurchaseRequisitionXML (string sXML)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Purchase Requisition.

- **Function AssignPONumberXML (lPRsAssigned As Long, lPOsCreated As Long, LastPO As Long, ByVal CreatePartVendor As Boolean, ByVal ApprovedBy As String) As Integer**
- **short AssignPONumberXML (ref object lPRsAssigned, ref object lPOsCreated, ref object lLastPO, bool CreatePartVendor, string ApprovedBy)**

Parameter	Description
lPRsAssigned	Used to return the number of Purchase Requisitions assigned.
lPOsCreated	Used to return the number of Purchase Orders that are created.
lLastPO	Returns the last PO number created.
CreatePartVendor	If True will automatically create a new Part Vendor relationship and process the order. If FALSE and there is not an existing Part Vendor relationship, the order will be skipped.
ApprovedBy	If a value is passed in for Approved By, only the orders that have a matching APRDBY value will be processed.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Assigns PO Numbers to Planned Orders and Purchase Requisitions with a valid Vendor ID and a Status of 1, 2 or 3.

- **Function ReleaseOrdersXML (ByVal StartPart As String, ByVal EndPart As String, ByVal StartBuyer As String, ByVal EndBuyer As String, ByVal StartOrder As String, ByVal EndOrder As String) As Integer**
- **short ReleaseOrdersXML (string StartPart, string EndPart, string StartBuyer, string EndBuyer, string StartOrder, string EndOrder)**

Parameter	Description
StartPart	Used to set the first part of a range. Set to blank for all parts within the order range.
EndPart	Used to set the last part of a range. Set to blank for all parts within the order range.
StartBuyer	Used to set the first buyer of a range. Set to blank for all buyers within the order range.
EndBuyer	Used to set the last buyer of a range. Set to blank for all buyers within the order range.
StartOrder	Set starting order for range.
EndOrder	Set ending order for range.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Converts approved Planned Orders into Purchase Requisitions. This routine performs the same processing as Print Purchase Requisitions.

- **Function AddNonInventoryPurchaseRequisitionXML (sXML As String, ByVal Description As String, sOrdNumOut As String) As Integer**
- **short AddNonInventoryPurchaseRequisitionXML (ref string sXML, string Description, ref string sOrdNumOut)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure.
Description	Description for NI Purchase Requisition Line Item
sOrdNumOut	Returns the Requisition number created

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Non-Inventory Purchase Requisition.

- **Function ChangeNonInventoryPurchaseRequisitionXML (sXML As String, ByVal Description As String) As Integer**
- **short ChangeNonInventoryPurchaseRequisitionXML (string sXML, string Description)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure.
Description	Description for NI Purchase Requisition Line Item

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Requisition.

- **Function DeleteNonInventoryPurchaseRequisitionXML (sXML As String) As Integer**
- **short DeleteNonInventoryPurchaseRequisitionXML (string sXML)**

Parameter	Description
sXML	Requires PurchaseRequisition_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Non-Inventory Purchase Requisition.

- **Function AssignNonInvPONumberXML (PRsAssigned As Long, POsCreated As Long, LastPO As Long, ByVal ApprovedBy As String) As Integer**
- **short AssignNonInvPONumberXML (ref int PRsAssigned, ref int POsCreated, ref int LastPO, string ApprovedBy)**

Parameter	Description
PRsAssigned	Used to return the number of Purchase Requisitions assigned.
PosCreated	Used to return the number of Purchase Orders that are created.
LastPO	Returns the last PO number created.
ApprovedBy	If a value is passed in for Approved By, only the orders that have a matching APPROV value will be processed.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Assigns PO Numbers to Purchase Requisitions with a valid Vendor ID.

4.3.7 Vendor Master

- **Function AddVendorXML (sXML As String) As Integer**
- **short AddVendorXML (string sXML)**

Parameter	Description
sXML	Requires Vendor_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Vendor.

- **Function ChangeVendorXML (sXML As String) As Integer**
- **short ChangeVendorXML (string sXML)**

Parameter	Description
sXML	Requires Vendor_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Vendor.

- **Function DeleteVendorXML (ByVal VenID As String) As Integer**
- **short DeleteVendorXML (string VenID)**

Parameter	Description
VenID	Vendor ID to delete.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Vendor.

4.3.8 Part Vendor

- **Function AddPartVendorXML (sXML As String) As Integer**
- **short AddPartVendorXML (string sXML)**

Parameter	Description
sXML	Requires Part_Vendor_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Part Vendor relationship.

- **Function ChangePartVendorXML (sXML As String) As Integer**
- **short ChangePartVendorXML (string sXML)**

Parameter	Description
sXML	Requires Part_Vendor_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Part Vendor relationship.

- **Function DeletePartVendorXML (ByVal VenID As String, ByVal PrtNum As String, ByVal MfgPart As String, ByVal MpnMfg As String) As Integer**
- **short DeletePartVendorXML (string VenID, string PrtNum, string MfgPart, string MpnMfg)**

Parameter	Description
VenID	Vendor ID
PrtNum	Part ID
MfgPart	Manufacturing Part Number.
MpnMfg	MPN Manufacturer

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Part Vendor relationship.

4.3.9 Puchase Part Data

- **Function ChangePurchasePartDataXML (sXML As String) As Integer**
- **short ChangePurchasePartDataXML (string sXML)**

Parameter	Description
sXML	Requires Part_Vendor_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes purchase part data for an existing MAX Part. Only the following fields can be changed: Purchasing planning lead time, buy lead time, stock lead time, buyer and recommended vendor.

4.3.10 Tax Codes

- **Function AddTaxCodeXML (sXML As String) As Integer**
- **short AddTaxCodeXML (string sXML)**

Parameter	Description
sXML	Requires Tax_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Tax code.

- **Function ChangeTaxCodeXML (sXML As String) As Integer**
- **short ChangeTaxCodeXML (string sXML)**

Parameter	Description
sXML	Requires Tax_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Tax code.

- **Function DeleteTaxCodeXML (sXML As String) As Integer**
- **short DeleteTaxCodeXML (string sXML)**

Parameter	Description
sXML	Requires Tax_Master_Table XML structure.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Tax code.

4.3.11 Shop Orders

- **Function AddUpdateShopOrderXML (sXML As String, ByRef sOrdNumOut As String) As Integer**
- **short AddUpdateShopOrderXML (string sXML, ref string sOrdNumOut)**

Parameter	Description
sXML	Requires ShopOrder_Table XML structure.
sOrdNumOut	Returns the Order Number created

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Either adds or updates an MAX Shop Order. If the order number is left blank an order will be added using the next available order number. The order will also be added if the order number is not blank and the order does not currently exist. Either MS or MF orders can be added (just set the TYPE_10 field to the appropriate type in the ShopOrder record). Both the default requirements and default routings will be created. The value of the SaveBillAndRouting property will determine if this function will create Requirement_Detail and Job_Progress records.

- **Function DeleteShopOrders (ByRef OrdNums() As String, ByRef *sErrMsgOut* As String) As Long**
- **int DeleteShopOrders (ref System.Array OrdNums, ref string sErrMsgOut)**

Parameter	Description
OrdNums()	String Array of order numbers to delete.
sErrMsgOut	Returns the text of the error message.

Returns
A long integer. Count of orders deleted.

Comments
Takes a string array of order numbers and returns the count of orders successfully deleted.

- **Function AddRoutingStepMAXORD (sOrdNum As String, sPrtNum As String, sOprSeq As String, sOprDes As String, sWorkcenter As String, dRunTime As Double, dSetupTime As Double, sOprID As String, sOprType As String, dQtyPer As Double, sHoldCode As String, sToolRef As String, sSetupType As String, IncludeStep As Integer, ByRef sErr As String) As Integer**
- **short AddRoutingStepMAXORD (string sOrdNum, string sPrtNum, string sOprSeq, string sOprDes, string sWorkcenter, double dRunTime, double dSetupTime, string sOprID, string sOprType, double dQtyPer, string sHoldCode, string sToolRef, string sSetupType, short nIncludeStep, ref string sErr)**

Parameter	Description
sOrdNum	Order Number to add the routing setup to
sPrtNum	PartNumber on Order
sOprSeq	Operation Sequence
sOprDes	Operation Description
sWorkcenter	Workcenter ID
dRunTime	Operation Run Time
dSetupTime	Operation Setup Time
sOprID	Operation ID
sOprType	Operation Type
dQtyPer	Quantity Per
sHoldCode	Hold Code
sToolRef	Tool Reference
sSetupType	Setup Type
nIncludeSetup	Include Setup flag 0=false, 1=true
sErr	Returns the text of the error message.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new routing step on a released shop order.

- **Function ChangeRoutingStepMAXORD (sOrdNum As String, sPrtNum As String, sOprSeq As String, sOprDes As String, sWorkcenter As String, dRunTime As Double, dSetupTime As Double, sOprID As String, sOprType As String, dQtyPer As Double, sHoldCode As String, sToolRef As String, sSetupType As String, IncludeStep As Integer, ByRef sErr As String) As Integer**
- **short ChangeRoutingStepMAXORD (string sOrdNum, string sPrtNum, string sOprSeq, string sOprDes, string sWorkcenter, double dRunTime, double dSetupTime, string sOprID, string sOprType, double dQtyPer, string sHoldCode, string sToolRef, string sSetupType, short nIncludeStep, ref string sErr)**

Parameter	Description
	Same as AddRoutingStepMAXORD parameters above

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing routing step on a released shop order.

- **Function DeleteRoutingStepMAXORD (sOrdNum As String, sOprSeq As String) As Integer**
- **short DeleteRoutingStepMAXORD (string sOrdNum, string sOprSeq)**

Parameter	Description
sOrdNum	Order Number
sOprSeq	Operation Sequence

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing routing step on a released shop order.

- **Function AddRequirementMAXORD (sOrdNum As String, sPrtNum As String, lCurDue As Long, dCurQty As Double, sStatus As String, dQtyPer As Double, iLTOffset As Integer, iScrap As Integer ByRef ErrMsg As String) As Integer**
- **short AddRequirementMAXORD (string sOrdNum, string sPrtNum, int lCurDue, double dCurQty, string sStatus, double dQtyPer, short iLTOffset, short iScrap, ref string ErrMsg)**

Parameter	Description
sOrdNum	Order Number
sPrtNum	Part Number
lCurDue	Current Due Date
dCurQty	Current Due Quantity
sStatus	Status of the Requirement (‘1’, ‘2’,. ‘3’, ‘4’, ‘5’)
dQtyPer	QuantityPer
iLTOffset	Lead Time Offset
iScrap	Planned Scrap
ErrMsg	Returns the text of the error message.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Adds a new Order Bill record on an order.

- **Function ChangeRequirementMAXORD (sOrdNum As String, sPrtNum As String, lCurDue As Long, dCurQty As Double, sStatus As String, dQtyPer As Double, iLTOffset As Integer, iScrap As Integer, ByRef ErrMsg As String) As Integer**
- **short ChangeRequirementMAXORD (string sOrdNum, string sPrtNum, int lCurDue, double dCurQty, string sStatus, double dQtyPer, short iLTOffset, short iScrap, ref string ErrMsg)**

Parameter	Description
	Same as AddRoutingStepMAXORD parameters above

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing Order Bill record on an order.

- **Function DeleteRequirementMAXORD (sOrdNum As String, sPrtNum As String, lCurDue As Long, ByRef ErrMsg As String) As Integer**
- **short DeleteRequirementMAXORD (string sOrdNum, string sPrtNum, int lCurDue, ref string ErrMsg)**

Parameter	Description
sOrdNum	Order Number
sPrtNum	Part Number
lCurDue	Current Due Date
ErrMsg	Returns the text of the error message.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing Order Bill record on an order.

- **Function DeletePartXML (PrtNum As String) As Integer**
- **short DeletePartXML (string PrtNum)**

Parameter	Description
PrtNum	Part Number

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing Part. With all correct validations to ensure related records are deleted first.

4.4 Interface to MAXTRAN2.DLL

- **Function ProcessTransXML (sXML As String, sErrMsg As String) As Long**
- **int ProcessTransXML (ref string sXML, ref string sErrMsg)**

Parameter	Description
sXML	Requires MAX_Transaction_Table XML structure.
sErrMsg	Returns the text of the error message.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Call to process MAX Transactions.

5. XML Table Structure

The XML structure uses the same field names and table names as currently defined in the SQL Database. The same XML structure is used by the ETL module.

The XML should be defined using version 1.0 specifications and utf-8 encoding.

The root element is always "eMAXExact", followed by the table name tag, the record name tag and then the actual fields. The basic outline should look like :

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <TableName_Table>
    <TableName>
…
    </TableName>
  </TableName_Table>
</eMAXExact>
```

The exact formatting of the XML Structure is fairly flexible as defined in the XML Spec, all the examples will be structured as above for readability, however passing the following XML will also work equally well:

```
<?xml version="1.0" encoding="utf-8" ?> <eMAXExact> <TableName_Table> <TableName> …
</TableName> </TableName_Table> </eMAXExact>
```

5.1 Formatting of XML Data

There are basically 3 data types that will be passed in the XML strings, dates and numbers. Be sure to check the table definitions either in SQL or the Database reference guide for the data type of a given field.

5.1.1 Passing Strings in XML

In MAX most fields do not allow null values so it is highly recommended to not pass an XML value that indicates null when calling the MAXUpdate functions. To indicate a blank string for the field DISPOSITION_39 either do not include the field at all in the XML or use the following are format:

```
<DISPOSITION_39></DISPOSITION_39>
```

Do not use the shortcut definition <DISPOSITION_39/> as this is interpreted as a null value and in many fields will cause an error to be returned.

5.1.2 Passing Dates in XML

In most fields requiring a date the format used should be yyyy-MM-dd, so February 3, 2011 would be 2011-02-03. The exception to this is the TNXDTE_39 field for processing transactions, this date should be passed in the format MMddyy, so the above example would be 020311. When a null date is required either do not pass the field in the XML or use 0001-01-01 as the date.

5.1.3 Passing Numbers in XML

For numeric fields in MAX simply pass the string equivalent of the number (in base 10 only, MAX does not recognize hex or octal numbering).

5.2 Sales Order Master XML Sample

This sample could be passed to the AddSalesOrderXML function:


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <SO_Master_Table>
    <SO_Master>
      <CUSTID_27>600</CUSTID_27>
      <GLXREF_27>000410000</GLXREF_27>
      <STYPE_27>CU</STYPE_27>
      <STATUS_27>3</STATUS_27>
      <ORDDTE_27>2012-01-12</ORDDTE_27>
      <REP1_27>MW01</REP1_27>
      <SPLIT1_27>100</SPLIT1_27>
      <TERMS_27>02</TERMS_27>
      <SHPVIA_27>01</SHPVIA_27>
      <FOB_27>Shipping Point</FOB_27>
      <TAXCD1_27>CHICITY</TAXCD1_27>
      <TAXCD2_27>COKCNTY</TAXCD2_27>
      <TAXCD3_27>ILSTATE</TAXCD3_27>
      <NAME_27>CRAZY EDDIE'S DISCOUNT STORE</NAME_27>
      <ADDR1_27>4534 W. MADISON ST.</ADDR1_27>
      <CITY_27>CHICAGO</CITY_27>
      <STATE_27>IL</STATE_27>
      <ZIPCD_27>60606</ZIPCD_27>
      <CNTRY_27>USA</CNTRY_27>
      <PHONE_27>312-438-6661</PHONE_27>
      <CNTCT_27>LOONEY LARRY</CNTCT_27>
      <FEDTAX_27>N</FEDTAX_27>
      <TAXABL_27>Y</TAXABL_27>
      <EXCRTE_27>1</EXCRTE_27>
      <FIXVAR_27>F</FIXVAR_27>
      <CURR_27>US</CURR_27>
      <TTAX_27>324</TTAX_27>
      <LNETAX_27>N</LNETAX_27>
    </SO_Master>
  </SO_Master_Table>
</eMAXExact>
```

5.3 Purchase Order Code XML Sample

The AddPurchaseOrderLineItemXML function allows creation of both header and detail records at the same time. In XML this is accomplished by having you pass a nested XML Structure containg both the Purchase_Order_Code and Order Master records as shown below:


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <Purchase_Order_Code_Table>
    <Purchase_Order_Code>
      <Order_Master>
        <ORDNUM_10>70001534</ORDNUM_10>
        <LINNUM_10>01</LINNUM_10>
        <DELNUM_10>01</DELNUM_10>
        <PRTNUM_10>13510</PRTNUM_10>
        <CURDUE_10>2011-02-12</CURDUE_10>
        <RECFLG_10>N</RECFLG_10>
        <TAXABLE_10>N</TAXABLE_10>
        <TYPE_10>PO</TYPE_10>
        <VENID_10>V100</VENID_10>
        <ORGDUE_10>2011-02-12</ORGDUE_10>
        <CURQTY_10>10</CURQTY_10>
        <ORGQTY_10>10</ORGQTY_10>
        <DUEQTY_10>10</DUEQTY_10>
        <FRMPLN_10>Y</FRMPLN_10>
        <STATUS_10>3</STATUS_10>
        <STK_10>RMI</STK_10>
        <PLANID_10>PL1</PLANID_10>
        <BUYER_10>BY1</BUYER_10>
        <REVLEV_10>A</REVLEV_10>
        <RECCOST_10>5</RECCOST_10>
        <PSCRAP_10>0</PSCRAP_10>
        <ASCRAP_10>0</ASCRAP_10>
        <SCRPCD_10>N</SCRPCD_10>
        <CSTCNV_10>1</CSTCNV_10>
        <TRNDTE_10>0001-01-01</TRNDTE_10>
        <NEGATV_10>N</NEGATV_10>
        <LABOR_10>0</LABOR_10>
        <REWORK_10>N</REWORK_10>
        <CRTSNS_10>N</CRTSNS_10>
        <TTLSNS_10>0</TTLSNS_10>
        <FORCUR_10>20.15</FORCUR_10>
        <EXCESS_10>0</EXCESS_10>
        <UOMCST_10>0</UOMCST_10>
        <UOMCNV_10>0</UOMCNV_10>
        <INSREQ_10>N</INSREQ_10>
        <CREDTE_10>2011-02-01</CREDTE_10>
        <COMCDE_10>COMP-PUR</COMCDE_10>
        <ORDPTP_10>B</ORDPTP_10>
        <JOBEXP_10>N</JOBEXP_10>
        <JOBCST_10>0</JOBCST_10>
        <UDFREF_10>MAXUpdate TEST</UDFREF_10>
        <DISC_10>-400</DISC_10>
        <DEXPFLG_10>N</DEXPFLG_10>
        <PLSTPRNT_10>N</PLSTPRNT_10>
        <ROUTPRNT_10>N</ROUTPRNT_10>
        <XDFINT_10>0</XDFINT_10>
```

```
      <XDFFLT_10>0</XDFFLT_10>
      <XDFDTE_10>2011-02-01</XDFDTE_10>
      <CreatedBy>MANAGER</CreatedBy>
      <ModifiedBy>MANAGER</ModifiedBy>
    </Order_Master>
  </Purchase_Order_Code>
 </Purchase_Order_Code_Table>
</eMAXExact>
```

5.4 Shop Order XML Sample

The AddUpdateShopOrderXML function is a little unusual in that the XML structure is based on a view in the SQL database rather than a physical table, however other than the fact it is based on a view there is no difference in how the XML is structured. Here is an example of that:


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <ShopOrder_Table>
    <ShopOrder>
      <ORDNUM_147>50007654</ORDNUM_147>
      <PRTNUM_147>11150</PRTNUM_147>
      <CURDUE_147>2011-09-29</CURDUE_147>
      <TYPE_147>MF</TYPE_147>
      <CURQTY_147>10</CURQTY_147>
      <FRMPLN_147>Y</FRMPLN_147>
      <STATUS_147>3</STATUS_147>
      <STK_147>FGI</STK_147>
      <CUSORD_147></CUSORD_147>
      <PLANID_147>PL1</PLANID_147>
      <ORDREF_147>MAXUpdate Test</ORDREF_147>
      <SCHFLG_147>Q</SCHFLG_147>
      <LOTNUM_147></LOTNUM_147>
      <BEGSER_147></BEGSER_147>
      <REWORK_147>N</REWORK_147>
      <FORCUR_147>127.345</FORCUR_147>
      <UDFKEY_147></UDFKEY_147>
      <UDFREF_147></UDFREF_147>
      <ALTBOM_147></ALTBOM_147>
      <ALTRTG_147></ALTRTG_147>
    </ShopOrder>
  </ShopOrder_Table>
</eMAXExact>
```

5.5 XML Structure for Transactions

Several samples of the XML used for the ProcessTransXML function will be included to show different methods of passing Lot/Serial information to the transactions as well as the formatting for some of the different transaction types.

5.5.1 Stock Transfer, part not under lot or serial control

Transfer 15 of part 13900 from RMI to QCI:

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>F</TYPE_39>
      <SUBTYPE_39> </SUBTYPE_39>
      <RCVSTK_39>QCI</RCVSTK_39>
      <ISSSTK_39>RMI</ISSSTK_39>
      <PRTNUM_39>13900</PRTNUM_39>
      <TNXQTY_39>15</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <UDFREF_39>TEST TRANSACTION</UDFREF_39>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```

5.5.2 Purchase Order Receipt part under lot control only

Receive 10 of part 13510 on PO 70001534-01-01 into RMI.

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>R</TYPE_39>
      <SUBTYPE_39>P</SUBTYPE_39>
      <TNXDTE_39>053111</TNXDTE_39>
      <TNXTME_39>165035</TNXTME_39>
      <RCVSTK_39>RMI</RCVSTK_39>
      <ORDNUM_39>70001534</ORDNUM_39>
      <LINNUM_39>01</LINNUM_39>
      <DELNUM_39>01</DELNUM_39>
      <TNXQTY_39>10</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <Lots>
        <Lot>
          <LOTNUM>Q3452</LOTNUM>
          <LOTQTY>8</LOTQTY>
        </Lot>
        <Lot>
          <LOTNUM>Q3453</LOTNUM>
          <LOTQTY>2</LOTQTY>
        </Lot>
      </Lots>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```

5.5.3 Shop Order Issue part under lot control only

Issue 5 of part 13510 to shop order 50007654 from lot Q3452.


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>I</TYPE_39>
      <SUBTYPE_39>S</SUBTYPE_39>
      <ISSSTK_39>RMI</ISSSTK_39>
      <PRTNUM_39>13510</PRTNUM_39>
      <ORDNUM_39>50007654</ORDNUM_39>
      <TNXQTY_39>5</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <UDFREF_39>TEST TRANSACTION</UDFREF_39>
      <LOT_39>Q3452</LOT_39>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```


5.5.4 Shop Order Receipt pat under lot and serial control

Receive 5 of part 11150 from shop order 50007654.


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>R</TYPE_39>
      <SUBTYPE_39>S</SUBTYPE_39>
      <ISSSTK_39>MSI</ISSSTK_39>
      <ORDNUM_39>50007654</ORDNUM_39>
      <TNXQTY_39>5</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <UDFREF_39>TEST TRANSACTION</UDFREF_39>
      <ASSCODE_39>R</ASSCODE_39>
      <LOT_39>Q3452</LOT_39>
      <SERIAL_39>S10001</SERIAL_39>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```

5.5.5 Cycle Count part under lot and serial control

Cycle Count part 11150 in location MSI. Lot/Serial information passed is the quantity in all lots after the transaction is complete.


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>C</TYPE_39>
      <SUBTYPE_39> </SUBTYPE_39>
      <ISSSTK_39>MSI</ISSSTK_39>
      <PRTNUM_39>11150</PRTNUM_39>
      <TNXQTY_39>5</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <UDFREF_39>TEST TRANSACTION</UDFREF_39>
      <ASSCODE_39>I</ASSCODE_39>
      <Lots>
        <Lot>
          <LOTNUM>L99443</LOTNUM>
          <LOTQTY>3</LOTQTY>
          <EXPDTE>2015-12-30</EXPDTE>
          <QCODE>False</QCODE>
          <DISPOSITION></DISPOSITION>
          <SerialNumbers>
            <SERIAL>S593001</SERIAL>
            <SERIAL>S593003</SERIAL>
            <SERIAL>S593067</SERIAL>
          </SerialNumbers>
        </Lot>
        <Lot>
          <LOTNUM>L99648</LOTNUM>
          <LOTQTY>2</LOTQTY>
          <EXPDTE>2015-01-31</EXPDTE>
          <QCODE>True</QCODE>
          <DISPOSITION>Q</DISPOSITION>
          <SerialNumbers>
            <SERIAL>S597021</SERIAL>
            <SERIAL>S597033</SERIAL>
          </SerialNumbers>
        </Lot>
      </Lots>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```

5.5.6 Adjust part under serial control only

Adjust 1 of part 12150 into MSI. To allow for the functionality of shifting parts between lots and serial numbers the Adjustment transaction requires the Lot/Serial information to be passed to be the total of all lot/serials in the stockroom at the end of the transaction. Also, this requires the Assignment Code (ASSCODE_39) to be I.


```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
  <MAX_Transaction_Table>
    <MAX_Transaction>
      <TYPE_39>A</TYPE_39>
      <SUBTYPE_39> </SUBTYPE_39>
      <ISSSTK_39>MSI</ISSSTK_39>
      <PRTNUM_39>12150</PRTNUM_39>
      <TNXQTY_39>1</TNXQTY_39>
      <USERNAME_39>MANAGER</USERNAME_39>
      <UDFREF_39>TEST TRANSACTION</UDFREF_39>
      <ASSCODE_39>I</ASSCODE_39>
      <SerialNumbers>
        <SERIAL>Y699345</SERIAL>
      </SerialNumbers>
    </MAX_Transaction>
  </MAX_Transaction_Table>
</eMAXExact>
```

6. Transaction Rules

6.1 General

Both Type and SubType are required for all transactions. If SubType is shown as blank, that means a blank space should be passed into the SubType ' ', not an empty string ''.

The Transaction Date and Transaction Time are optional fields, the current date/time of the system will be used if they are not passed in the XML. Usually it is best to leave these out of the XML since even if passed they will only be used if the system switch "Trans Date Time" is set to Y.

In all cases the Lot/Serial information must be passed using quantities in the BOM UOM for the part. Some transactions (see notes in Sections 6.7.x to identify transactions that require a UOM other than the BOM UOM) have a transaction quantity in the Purchasing UOM, even in these cases the Lot/Serial information must be in BOM UOM as this is the UOM used by MAX for all stock information.

6.2 Lot/Serial Tracking (XML)

Lot/Serial information must be passed in transactions for all parts under Lot/Serial control. The XML examples in the previous section show the different ways this information can be passed. For a part under Serial control the Assignment Code field (ASSCODE_39) must be **I** (**I**ndividual), or **R** (**R**ange). If **R** is selected it indicates the serial numbers will be assigned sequentially starting with the serial number passed in SERIAL_39. The Serial number passed must follow the standard MAX rules for auto assigned serial numbers. If **I** is selected it indicates the individual serial numbers will be contained in the <SerialNumbers> section of the XML.

For Cycle Count and zero quantity Adjustment transactions the Lot/Serial information passed must contain the total Lot/Serial information for the stockroom being transacted. This is not a delta quantity, the entire on-hand quantity for all lots/serials after the transaction completes must be passed in the XML. In these cases the **R** Assignment code is not recommended as it is likely to cause unexpected results.

6.3 Lot/Serial Tracking (Data Structures)

Either the lot number and/or the serial number field must be filled in. If serial tracking is used the Assignment Code for Serial numbers must be **I** (**I**ndividual), or **R** (**R**ange) in the serial number field. If **R** is selected the serial number for the first serial number should be entered in the serial number field - the remaining serial numbers will be automatically generated using the transaction quantity (only serial numbers ending in a numeric value can be used for the range option). The **I** option requires that an array of serial numbers be passed. The method for this is as follows:

6.3.1 Serial Tracking (Individual Option)

Three steps are required in addition to making the transaction call. The first step is to call **CreateSerialArray** with the number of serial records to be created. This allocates enough memory to hold the serial numbers. The next step is to call **SetSerialNumber** for each serial number to be created. Then call **ProcessTrans** to do the desired transaction. The last step is to call **DeleteSerialArray** to free up the memory allocated by **CreateSerialArray**. Be sure to set the "Assignment code for serial #'s" field to 'I' for individual.

6.3.2 Lot and Serial Tracking (Individual Option)

These are handles as above except using the functions **CreateLotAndSerialArray**, **SetLotAndSerialNumber** and **DeleteLotAndSerialArray**.

6.3.3 Cycle Count (Subtype L)

This applies to cycle counts with lots only. Cycle counts with serial numbers only use the procedure above. Cycle counts with Lot and Serials can not be done using this procedure unless the “R” option is used for serial numbers.

As for individual serial numbers, three steps are required in addition to making the transaction call. The first step is to call **CreateLotQtyArray** with the number of lot records to be created. This allocates enough memory to hold the lot numbers. The next step is to call **SetLotQty** for each lot number to be created. Then call **ProcessTrans** to do the cycle count transaction. The last step is to call **DeleteLotQtyArray** to free up the memory allocated by **CreateLotQtyArray**.

6.4 Supported Transactions

Type	Sub Type	Description
A		Adjustment
A	P	Adjustment – Physical Inventory
B		Post-Production Completion (Repetitive)
C		Cycle Count
C	L	Cycle Count Lot Part
F		Stock Transfer
F	R	Consignment Return
F	S	Consignment Shipment
I	O	Issue – Subcontract
I	S	Issue – Shop Order
I	U	Issue – Unplanned
L	D	Labor Logout Indirect
L	I	Labor Login Indirect
L	L	Labor Login
L	O	Labor Logout
P	C	Post Operation Completion
P	T	Post Time Ticket Completion
R	B	Receipt – Backflush
R	N	Receipt – Non-Inventory
R	O	Receipt – Subcontract Order
R	P	Receipt – Purchase Order
R	S	Receipt – Shop Order
R	U	Receipt – Unplanned or RMA
S		Shipment
S	M	Shipment – Subcontract BOM
S	O	Shipment – Sales Order
T		Time Ticket Data
W		Rework
Y		Scrap
Y	O	Scrap – Subcontract
Y	R	Scrap – Repetitive

6.5 Supported Orders

This shows the type code stored in various tables for different order types. See notes for actual table names.

Description	OM[1]	RD[2]	SOM[3]	SOD[4]	POC[5]	RMA[6]
Quote			QT	QT		
Sales Order	CU	CU	CU	CU		
Credit Memo			CR	CR		
Consignment Order	CS	CS	CS	CS		
RMA						*
Purchase Requisition	PR					
Purchase Order	PO				PO	
Subcontract Requisition	SR					
Subcontract Order	SO	RQ			SO	
Vendor Order	VO				VO	
Non-Inventory PR	NI					
Non-Inventory PO	NI				NI	
Master Schedule Order	MS	RQ				
Shop Order	MF	RQ				
Planned Order	PL	RQ				
Forecast Order	FC	FC				

[1] Order_Master
[2] Requirement_Detail
[3] SO_Master
[4] SO_Detail
[5] Purchase_Order_Code
[6] RMA_Detail

6.6 Transaction Descriptions

Transaction Name	Type	Size	Description
RECEIVE TO STOCK ID	A	8	Used in Receipt to stock and transfer stock to stock transactions. Must be a valid stock location in the stock master file. If this field is blank and required for a particular transaction, the primary stock location for the part being transacted will be used.
ISSUE/TRANSFER FROM STOCK ID	A	8	Must be a valid stock location with enough quantity on hand in that location to perform the issue/transfer. If this field is left blank, the primary stock location for the part ID being loaded will be used.
PART ID	A	30	Must be valid if required by operation type / subtype. Non inventory transactions (shipments) allow any part number.
ORDER #	A	8	If required by transaction type and subtype.
LINE #	A	2	If required by transaction type and subtype.
DELIVERY #	A	2	If required by transaction type and subtype.
TRANSACTION QTY	N	8	Any number between -999999 and 999999 depending on the type of transaction. Negative receipts to stock are allowed.
TRANSACTION DATE	A	6	Will use the value contained in this field if not empty and is a valid date. If this field is empty, it will use the current system date.
TRANSACTION TIME	A	6	Will use the value contained in this field if not empty and is a valid time based on the 24-hour clock. I.E. 175000 is 5:50:00 pm. If this field is empty, it will use the current system time.
REFERENCE DESCRIPTION	A	20	User defined
GL REFERENCE	A	3	3 character override must be valid in Maintain GL override accounts.
OPERATION SEQUENCE #	A	4	If required by transaction type and subtype. Must be a valid operation sequence for the order specified.
NEXT OPERATION SEQUENCE #	A	4	If required by transaction type and subtype. Must be a valid operation sequence for the order specified.
ACTUAL RUN TIME	N	8	In hours. 1.5 = 1 hour 30 minutes
ACTUAL SETUP TIME	N	8	In hours. 1.5 = 1 hour 30 minutes
SHIFT	I	2	1, 2, or 3
Zone	A	10	User defined will pull from prtmst->zone if blank and prtmst not empty.
EMPLOYEE ID	A	7	Must be valid employee id (if required by transaction type).
TICKET #	A	6	
START TIME (hhmm)	A	4	
END TIME (hhmm)	A	4	
SETUP TIME	N	8	In hours. 1.5 = 1 hour 30 minutes
ELAPSED TIME	N	8	In hours. 1.5 = 1 hour 30 minutes
TIME TICKET ACTUAL SCRAP QTY	N	8	
REASON CODE	A	1	
USER NAME	A	100	Must be a valid User Name.
USER DEFINED KEY	A	15	
USER DEFINED REFERENCE	A	25	
ASSIGNMENT CODE FOR SERIAL #'s	A	1	Should be N, I, or R if serial numbers are desired, blank otherwise. Use I with CreateSerialArray described previously.
LOT NUMBER	A	30	
SERIAL NUMBER	A	30	
TERMINAL ID	A	15	Usually used only in DOS data logging applications.
QCODE	A	1	QCODE should be Y or N for yes or no.
EXPDATE	*A*	*6*	Expiration Date in format YYMMDD.
DEFECT	A	4	Defect code.
RECPL	A	1	
DISPOSITION	A	15	Lot Disposition.
CLASS	A	159	QuickBooks ONLY, QB Class Code

6.7 Required Fields Matrix

Transaction Type	TYPE_39	**A**	**A**	**B**	**C**	**C**	**F**	**F**	**F**	**I**	**I**	**I**	**L**	**L**	**L**	**L**
Transaction Sub Type	SUBTYPE_39		**P**			**L**		**R**	**S**	**O**	**S**	**U**	**D**	**I**	**L**	**O**
Transaction Date	TNXDTE_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Transaction Time	TNXTME_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Order Number	ORDNUM_39							✓	✓	✓	✓	O			✓	✓
Line Number	LINNUM_39							✓	✓	✓	R	O				
Delivery Number	DELNUM_39							✓	✓	✓	R	O				
Operation Sequence	OPRSEQ_39														✓	✓
Next Operation Seq.	NXTOPR_39															
Part Identifier	PRTNUM_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓				
Issue From Stock ID	ISSSTK_39	✓	✓	O	✓	✓	✓		✓	✓	✓	✓				
Receive to Stock ID	RCVSTK_39			✓			✓	✓								
Zone	LOCATOR_39	O	O		O	O	O									
Transaction Quantity	TNXQTY_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓				✓
Reference Description	REFDSC_39	O	O	O	O	O	O			O	O	O				
GL Acct. Override	GLREF_39	O	O		O	O	O			O	O	O		✓		
Shift	SHIFT_39												✓	✓		
Employee ID	EMPID_39												✓	✓	✓	✓
Ticket Number	TICKET_39															✓
Start Time	STARTTIME_39															
End Time	ENDTIME_39															
Setup Time	SETUPTIME_39															
Elapsed Time	ELAPSED_39															
Actual Run Time	RUNACT_39															✓
Actual Setup Time	SETACT_39															O
Time Ticket Scrap Qty	ASCRAP_39															O
Reason Code	REASON_39															
User Name	USERNAME_39	O	O	O	O	O	O	O	O	O	O	O				
User Defined Key	UDFKEY_39	O	O	O	O	O	O	O	O	O	O	O				
User Defined Reference	UDFREF_39	O	O	O	O	O	O	O	O	O	O	O				
Serial # Assign Code	ASSCODE_39	O	O		O	O	O	O	O	O	O	O				
Lot Number	LOT_39	O	O		O	O	O	O	O	O	O	O				
Serial Number	SERIAL_39	O	O		O	O	O	O	O	O	O	O				
Terminal ID	TERMINAL_39															
Qcode	QCODE_39	O	O		O	O	O	O	O	O	O	O				
Expdate	EXPDATE_39	O	O		O	O	O	O	O	O	O	O				✓
Defect	DEFECT_39	O	O		O	O	O	O	O	O	O	O				O
Recpl	RECPL_39	O	O		O	O	O	O	O	O	O	O				
Disposition	DISPOSITION_39	O	O		O	O	O	O	O	O	O	O				

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

Required Fields Matrix (Cont'd)

Transaction Type	TYPE_39	**P**	**P**	**R**	**R**	**R**	**R**	**R**	**R**	**S**	**S**	**S**	**T**	**W**	**Y**	**Y**	**Y**
Transaction Sub Type	SUBTYPE_39	**C**	**T**	**B**	**N**	**O**	**P**	**S**	**U**		**M**	**O**				**O**	**R**
Transaction Date	TNXDTE_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Transaction Time	TNXTME_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Order Number	ORDNUM_39	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	O
Line Number	LINNUM_39				✓	✓	✓			✓	✓	✓		✓		✓	O
Delivery Number	DELNUM_39					✓	✓			✓	✓	✓		✓		✓	O
Operation Sequence	OPRSEQ_39	✓	✓			✓							✓	✓	✓	✓	
Next Operation Seq.	NXTOPR_39	O												O			
Part Identifier	PRTNUM_39								✓							✓	✓
Issue From Stock ID	ISSSTK_39									✓	✓	✓					✓
Receive to Stock ID	RCVSTK_39			O		O	O	O	✓								
Zone	LOCATOR_39			O	O	O	O	O	O				O			O	
Transaction Quantity	TNXQTY_39	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Reference Description	REFDSC_39	O		O		O	O	O	O	O	O	O		O	O	O	O
GL Acct. Override	GLREF_39			O	O	O	O	O	O	O	O	O		O	O	O	O
Shift	SHIFT_39	✓	O			✓							✓		O	O	
Employee ID	EMPID_39		✓										✓				
Ticket Number	TICKET_39		✓										✓				
Start Time	STARTTIME_39		✓										✓				
End Time	ENDTIME_39		O										✓				
Setup Time	SETUPTIME_39		O										O				
Elapsed Time	ELAPSED_39		O										O				
Actual Run Time	RUNACT_39	✓	✓										✓				
Actual Setup Time	SETACT_39	✓	✓										✓				
Time Ticket Scrap Qty	ASCRAP_39		✓										✓				
Reason Code	REASON_39		O													O	
User Name	USERNAME_39	O	O	O	O	O	O	O	O	O	O	O			O	O	O
User Defined Key	UDFKEY_39	O	O	O	O	O	O	O	O	O	O	O		O	O	O	O
User Defined Reference	UDFREF_39	O	O	O	O	O	O	O	O	O	O	O		O	O	O	O
Serial # Assign Code	ASSCODE_39			O		O	O	O	O	O	O	O					O
Lot Number	LOT_39			O		O	O	O	O	O	O	O					O
Serial Number	SERIAL_39			O		O	O	O	O	O	O	O					O
Terminal ID	TERMINAL_39																
Qcode	QCODE_39			O		O	O	O	O	O	O	O					O
Expdate	EXPDATE_39		✓	O		O	O	O	O	O	O	O	✓				O
Defect	DEFECT_39			O		O	O	O	O	O	O	O				O	O
Recpl	RECPL_39			O		O	O	O	O	O	O	O					O
Disposition	DISPOSITION_39			O		O	O	O	O	O	O	O					O

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.1 Adjustment

Transaction Type	A	Comments
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone	O	If blank, the default zone used
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.2 Adjustment – Physical Inventory

Transaction Type	**A**	**Comments**
Transaction Sub Type	**P**	Updates Cycle Count Date not transaction date
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone	O	If blank, the default zone used
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.3 Post-Production Completion (Repetative)

Transaction Type	B	Comments
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	O	Used as Alternate StockID
Receive to Stock ID	✓	
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override		
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.4 Cycle Count

Transaction Type	**C**	**Comments**
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone	O	If blank, uses default part zone.
Transaction Quantity	✓	Must be within % and $ tolerance.
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.5 Cycle Count Lot Part

Transaction Type	**C**	**Comments**
Transaction Sub Type	**L**	
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone	O	If blank, uses default part zone.
Transaction Quantity	✓	Must be within % and $ tolerance.
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.6 Stock Transfer

Transaction Type	**F**	**Comments**
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID	✓	
Zone	O	**FROM** Side: Zone from PRTSTK
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code	O	
Expiration Date	O	
Defect Code	O	
Recpl	O	
Lot Disposition	O	

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.7 Consignment Return

Transaction Type	**F**	**Comments**
Transaction Sub Type	R	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID		
Receive to Stock ID	✓	
Zone	O	**From** Side: Zone from PRTSTK
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code	O	
Expiration Date	O	
Defect Code	O	
Recpl	O	
Lot Disposition	O	

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.8 Consignment Shipment

Transaction Type	**F**	**Comments**
Transaction Sub Type	S	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone	O	**FROM** Side: Zone from PRTSTK
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code	O	
Expiration Date	O	
Defect Code	O	
Recpl	O	
Lot Disposition	O	

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.9 Issue – Subcontract

Transaction Type	**I**	**Comments**
Transaction Sub Type	**O**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		Uses Zone from PRTSTK record.
Transaction Quantity	✓	Must be within % & $ tolerance.
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.10 Issue – Shop Order

Transaction Type	**I**	**Comments**
Transaction Sub Type	**S**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	R	Set to '00' if blank.
Delivery Number	R	Set to '00' if blank.
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		Uses Zone from PRTSTK record.
Transaction Quantity	✓	Must be within % & $ tolerance.
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.11 Issue – Unplanned

Transaction Type	**I**	**Comments**
Transaction Sub Type	**U**	
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		Uses Zone from the PRTSTK record.
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

.

6.7.12 Labor Logout Indirect

Transaction Type	**L**	**Comments**
Transaction Sub Type	**D**	
Transaction Date	✓	Logout Date
Transaction Time	✓	Logout Time
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity		
Reference Description		
GL Acct. Override		
Shift	✓	
Employee ID	✓	
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key		
User Defined Reference		
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used

R = Recommend

6.7.13 Labor Login Indirect

Transaction Type	**L**	**Comments**
Transaction Sub Type	**I**	
Transaction Date	✓	Login Date
Transaction Time	✓	Login Time
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity		
Reference Description		
GL Acct. Override	✓	Indirect Acct Code
Shift	✓	
Employee ID	✓	
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key		
User Defined Reference		
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used

R = Recommend

6.7.14 Labor Login

Transaction Type	**L**	**Comments**
Transaction Sub Type	**L**	
Transaction Date	✓	Login Date
Transaction Time	✓	Login Time
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence	✓	
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity		
Reference Description		
GL Acct. Override		
Shift		
Employee ID	✓	
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key		
User Defined Reference		
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓ = Required Field
O = Optional Field

Blank	=	Not Used
R	=	Recommend (but not required)

6.7.15 Labor Logout

Transaction Type	**L**	**Comments**
Transaction Sub Type	**O**	
Transaction Date	✓	Logout Date
Transaction Time	✓	Logout Time
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence	✓	
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description		
GL Acct. Override		
Shift		
Employee ID	✓	
Ticket Number	✓	Login Time goes here (not Ticket #)
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time	✓	
Actual Setup Time	O	
Time Ticket Scrap Qty	O	Scrap Qty
Reason Code		
User Name		
User Defined Key		
User Defined Reference		
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		
Quarantine Code		
Expiration Date	✓	Login Date goes here (not Exp Date)
Defect Code	O	
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used

R	=	Recommend (but not required)

6.7.16 Post Operation Completion

Transaction Type	**P**	**Comments**
Transaction Sub Type	**C**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence	✓	
Next Operation Sequence	O	
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override		
Shift	✓	Must be between '1' and '3'.
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time	✓	
Actual Setup Time	✓	
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.17 Post Time Ticket Completion

Transaction Type	P	Comments
Transaction Sub Type	**T**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	If 1st byte = 'I', Indirect Labor.
Line Number		
Delivery Number		
Operation Sequence	✓	If Indirect Labor, bytes 1 - 3 become the Acct. type in the EMPWRK record.
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity	✓	Must be > 0.
Reference Description		
GL Acct. Override		
Shift	O	
Employee ID	✓	
Ticket Number	✓	
Start Time	✓	
End Time	O	
Setup Time	O	
Elapsed Time	O	
Actual Run Time	✓	
Actual Setup Time	✓	
Time Ticket Scrap Qty	✓	
Reason Code	O	
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date	✓	Holds the time ticket date.
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.18 Receipt – Backflush

Transaction Type	**R**	**Comments**
Transaction Sub Type	**B**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID	O	If blank, use the default Stockroom for the part.
Zone	O	
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.19 Receipt – Non-Inventory

Transaction Type	**R**	**Comments**
Transaction Sub Type	**N**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone	O	
Transaction Quantity	✓	
Reference Description		
GL Acct. Override	O	Required if MAX GLREF Required Flag is set to 'Y'.
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.20 Receipt – Subcontract Order or Vendor Order

Transaction Type	**R**	**Comments**
Transaction Sub Type	**O**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence	✓	
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID	O	If blank, use the default stockroom for the part.
Zone	O	If blank, use the default zone for the part.
Transaction Quantity	✓	Type SO use Purchasing UOM
Reference Description	O	
GL Acct. Override	O	
Shift	✓	
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.21 Receipt – Purchase Order

Transaction Type	**R**	**Comments**
Transaction Sub Type	**P**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID	O	If blank, use the default stockroom for the part.
Zone	O	If blank, use the default zone for the part.
Transaction Quantity	✓	In Purchasing UOM
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.22 Receipt – Shop Order

Transaction Type	**R**	**Comments**
Transaction Sub Type	**S**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID	O	If blank, use the default stockroom for the part.
Zone	O	If blank, use the default zone for the part.
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.23 Receipt - Unplanned

Transaction Type	**R**	**Comments**
Transaction Sub Type	**U**	
Transaction Date	✓	
Transaction Time	✓	
Order Number		
Line Number		
Delivery Number		
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID		
Receive to Stock ID	✓	I
Zone	O	If blank, use the default zone for the part.
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.24 Shipment

Transaction Type	S	Comments
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.25 Shipment – Subcontract Order

Transaction Type	**S**	**Comments**
Transaction Sub Type	**M**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.26 Shipment – Sales Order

Transaction Type	S	Comments
Transaction Sub Type	**O**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence		
Next Operation Sequence		
Part Identifier		
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.27 Time Ticket

Transaction Type	**T**	**Comments**
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence	✓	First byte can be Indirect Labor
Next Operation Sequence		Acct Type
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone	O	**TO** Side: default Zone from par,
Transaction Quantity	✓	
Reference Description		
GL Acct. Override		
Shift	✓	
Employee ID	✓	
Ticket Number	✓	
Start Time	✓	
End Time	✓	
Setup Time	O	
Elapsed Time	O	
Actual Run Time	✓	
Actual Setup Time	✓	
Time Ticket Scrap Qty	✓	
Reason Code		
User Name		
User Defined Key		
User Defined Reference		
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		
Quarantine Code		
Expiration Date	✓	Holds the time ticket date.
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used

R	=	Recommend (but not required)

6.7.28 Rework

Transaction Type	**W**	**Comments**
Transaction Sub Type		**Negative Post Op Completion**
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence	✓	Current Operation
Next Operation Sequence	O	Operation which needs to be reworked. If blank, backs out quantity to the first operation.
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity	✓	If blank assumes the entire order quantity.
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name		
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.29 Scrap

Transaction Type	**Y**	**Comments**
Transaction Sub Type		
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number		
Delivery Number		
Operation Sequence	✓	
Next Operation Sequence		
Part Identifier		
Issue From Stock ID		
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift	O	
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code		
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.30 Scrap – Subcontract

Transaction Type	**Y**	**Comments**
Transaction Sub Type	**O**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	✓	
Line Number	✓	
Delivery Number	✓	
Operation Sequence	✓	
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID		
Receive to Stock ID		
Zone	O	
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift	O	
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code	O	
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s		
Lot Number		
Serial Number		
Terminal ID		
Quarantine Code		
Expiration Date		
Defect Code	O	Required per System Switch
Recpl		
Lot Disposition		

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

6.7.31 Scrap – Repetative

Transaction Type	**Y**	**Comments**
Transaction Sub Type	**R**	
Transaction Date	✓	
Transaction Time	✓	
Order Number	O	
Line Number	O	
Delivery Number	O	
Operation Sequence		
Next Operation Sequence		
Part Identifier	✓	
Issue From Stock ID	✓	
Receive to Stock ID		
Zone		
Transaction Quantity	✓	
Reference Description	O	
GL Acct. Override	O	
Shift		
Employee ID		
Ticket Number		
Start Time		
End Time		
Setup Time		
Elapsed Time		
Actual Run Time		
Actual Setup Time		
Time Ticket Scrap Qty		
Reason Code		
User Name	O	
User Defined Key	O	
User Defined Reference	O	
Assignment Code for Serial #'s	O	
Lot Number	O	
Serial Number	O	
Terminal ID		
Quarantine Code	O	
Expiration Date	O	
Defect Code	O	Required per System Switch
Recpl	O	
Lot Disposition	O	

Legend

✓	=	Required Field
O	=	Optional Field
Blank	=	Not Used
R	=	Recommend (but not required)

7. Usage

To use the MAX Update package for development, the following steps should be taken:

1. We recommend VB.NET or C#.NET as the development language.
2. Make sure the MAX client is installed on the development machine and add the RMClient\EFW folder into your system PATH statement.
3. In the .NET environment add ExactRMEnc.DLL, ERMRemCl.DLL and MaxUpdateXML.DLL as references to your project. (ExactRMEnc and ERMRemCl are not directly called, but adding them as references will cause them to automatically copy to the EXE directory allowing Debugging to work easily).
4. Copy the .dll, and the appropriate include files to your development directory. For the .NET environment the above step handles this automatically.

   In the VB6 environment the ExactRMEnc.DLL and ERMRemCl.DLL must be copied into the C:\Program Files\Microsoft Visual Studio\VB98 directory to allow debugging from within the development environment.

   For other environments the DLLs must be in the directory of the running EXE in order for MAXUpdate calls to work correctly.
5. If you are developing in C, you can use the header files directly without modification. Visual Basic developers will be able to use the .str files directly, but will need to write DECLARE statements for the functions they will be calling. C++ programmers must use the "wrapper" functions described above in the *Function Description*.

   If you are developing from another environment, you will need to declare the above functions according to the rules of that environment. The functions are for (32-bit) C-style calls with left-to-right parameter passing. The called function takes care of stack clean up.

Note to developers: Repeated use of the Wrapper is best done by minimizing the number of object creation instances. After creating the object, any number of function calls can be made. Conversely, repeatedly creating and destroying the Wrapper object between calls can result in the following error: Error 91: "Object variable or With block variable not set".

In the ASP environment, developers can choose between application, session or global object based on their application and single or multi-user environment. Generally, we recommend a session object to avoid transaction conflicts and deadlocks.

8. VB6 Programming with MAXUpdate.

8.1 Introduction

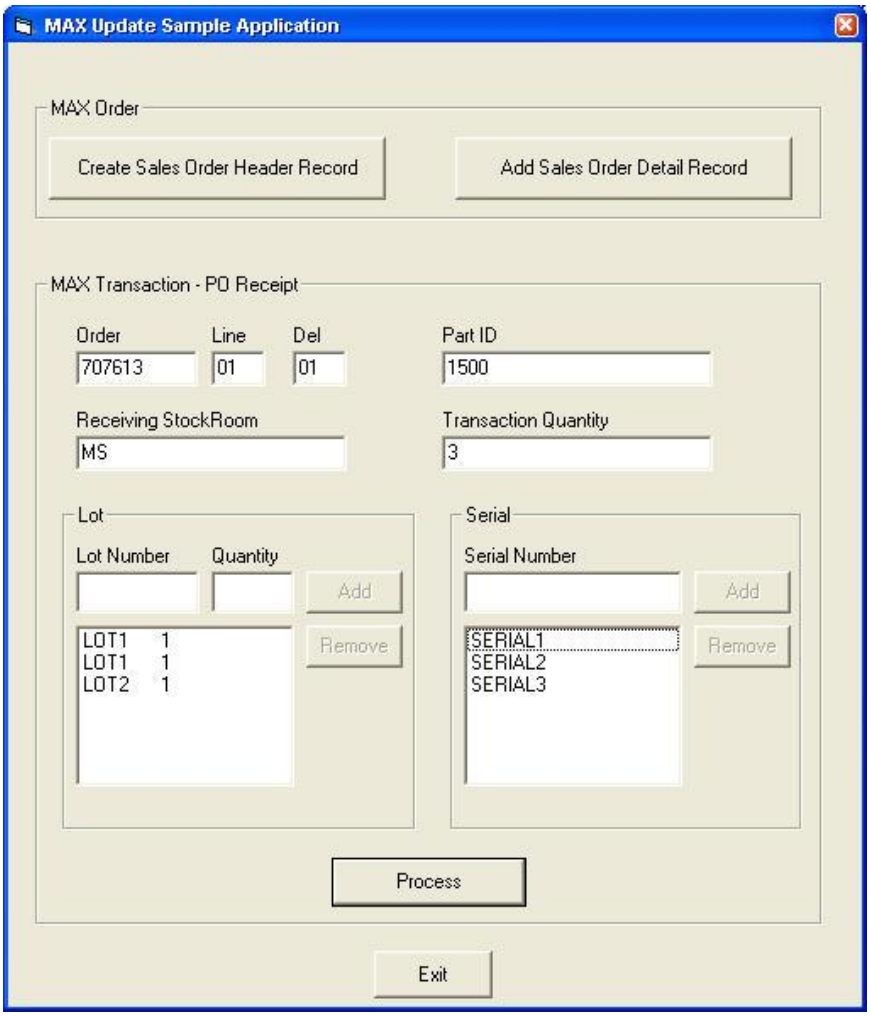
Microsoft Visual Basic, Access, Excel and Word are examples of applications that support the Visual Basic for applications programming language. They are all applications that contain the necessary programming tools to call Dynamic Link Libraries (DLLs).

The primary purpose of MAXUpdate in a Visual Basic application is to simplify the programming procedure necessary to update your MAX database. Even with a minimal understanding of the file relationships and data structures in MAX, you can successfully perform complex tasks.

MAXUpdate also provides the necessary control to ensure data integrity, and it provides the Visual Basic developer with the assurance that his or her application follows all data relationship rules required in MAX .

The first step in developing your application is to determine the type of MAX data updates you will perform. With this in mind, you can determine if you need only one of the libraries or both. When developing your application, only include the library that you will use. For example, including MAXORDR2.DLL in your application when you plan on performing only inventory transactions is unnecessary. In this case only include the MAXTRAN2.DLL etc.

MAXUpdate is primarily a tool used to create user interfaces that add, update and delete data in the MAX database. You will however also probably have the need to retrieve and display data from the MAX database. Only limited data retrieval capabilities exist in the MAXUpdate Dynamic Link Libraries. There are several ways to retrieve data outside MAXUpdate, and you should choose the method that you feel the most comfortable with and that provides adequate performance. Also, note that any method you use to retrieve data from MAX should not create table or record locks on the database, unexpected behavior such as lockups and timeouts of the MAX Client on other workstations can be caused by a custom application locking records with an improperly designed database query.



8.2 Writing your application

This section describes the steps necessary to create the Visual Basic application as shown above, using the MAXUpdate DLLs with Visual Basic version 6.0. The examples shown can be adapted for use in other development tools that support Visual Basic for Applications. The purpose of the sample application is to demonstrate how to add data via MAXUpdate.

Start your new Visual Basic project, with at least one form.

Next add a module, to hold MAXUpdate specific code.

Supplied with MAXUpdate are low level code files for two MAX DLLs, MAXORDER32.BAS and MAXTRANS32.BAS. These contain all the function and data type definitions necessary to use each of the modules. Either or both of these must be included in your project. If you are only dealing with transactions, then MAXTRANS32 will be sufficient. All other updates are dealt with by means of functions in MAXORDER32.

In order to run your application, you will need to have access to other DLLs on which MAXTRAN2 and MAXORDR2 depend. To obtain an up-to-date list of the dependent DLLs, it is recommended that you open MAXTRAN2 and MAXORDR2 with Quikview and search for ".DLL". Currently this list consists of:

MFC42.DLL
MSVCRT.DLL
MFC042.DLL
MFVCIRT.DLL
MAXEXEC2.DLL

MAXTRAN2, MAXORDR2 and MAXEXEC2 are supplied with MAX , and will be found under the folder to which it has been installed. The remaining DLLs will be found in your EFW directory.

DLLs must be initialized before using, and you must obtain an object handle for each. The easiest place to do this is at the start of your application, for example when you load the first form. See the sample code provided with MAXUpdate.

9. C++ Programming with MAXUpdate.

See section 7.1 and 7.2 above. The same principles apply. The following header files have been supplied to aid in programming:

Include File	Description
ITrans.h	Structure definitions and defines used with the MAX Trans DLL Use the Itrans structure for passing data to the DLL.
Maxorder.ph	Prototypes for MAX Order interfaces.
Maxtrans.ph	Prototypes for MAX Transactions interfaces.
Maxdefs.h	Commonly used MAX definitions

10. Programming ETL with MAXUpdate.NET

The MAX ETL Application now included in Core MAX allows a user to Download (Extract) and Upload (Load) MAX Data to and from XML tagged files, using the provided User Interface and/or Command line functionality as documented in the MAX ETL User Guide and help.

10.1.1 Introduction I – What MAX Customers Get for Free

The two main components used by the ETL application, 1) MAXINTBO.DLL and 2) the MaxDataSchema.xsd file are now included in core MAX, with MAX batch 1591 and higher.

We expose the Extract and Load functionality to MAX Programmers and Power Users in MAXINTBO.DLL, a .NET DLL that exposes interfaces the programmer can use to call and perform the extract or load operation.

The MaxDataSchema.xsd file describes the XML Schema for the data extracted and loaded by MAX ETL.

10.1.2 Introduction II - What MAXUpdate Users Pay For

MAX Programmers and Power users pay extra for MAXUpdate.

MAXUpdate now includes sample Extract and Load application code (MAXETLTst.exe) and the following sections of this reference guide which describe this sample application as well as how to create a new Extract\Load application in a .NET programming environment.

10.2 Using MAXETLTst.exe the Extract\Load Sample Application

The sample application (MAXETLTst.exe) demonstrates how XML data can be Extracted and Loaded to MAX using the MAXINTBO.DLL.

After opening the MAXETLTst.exe application, *but before* running a download/upload option, you must:

1) Set the RM Server address. This is the SQL server that is hosting the ExactRM Database.
2) Set the Server Password. This is SQL Server password.
3) Set the RM License path. This is the path in which the ExactRM.lic is located.
4) Click Get Companies to choose the RM Company you wish to connect to.
5) Choose the MAX Data Schema file and
6) Choose the Log file to hold the status messages.

Also, MAXINTBO.DLL is the default reference to the sample application. IF YOU RUN MAX IN A NON-STANDARD NETWORK PATH, YOU MUST RE-SET MAXINTBO.DLL as the default reference to the sample application. The Standard (default) MAX Network path is C:\EXACT\RMCLIENT\EFW.

Four distinct scenarios follow which explain the MAX ETL upload/download functions in detail.

1. Download all Records from an MAX Table into an XML File.

Figure 1. shows the dialog where the user selects a table to be downloaded. The Output file is <TableName>.xml. Clicking the “Download All” button downloads all the records from the Table into the output file.

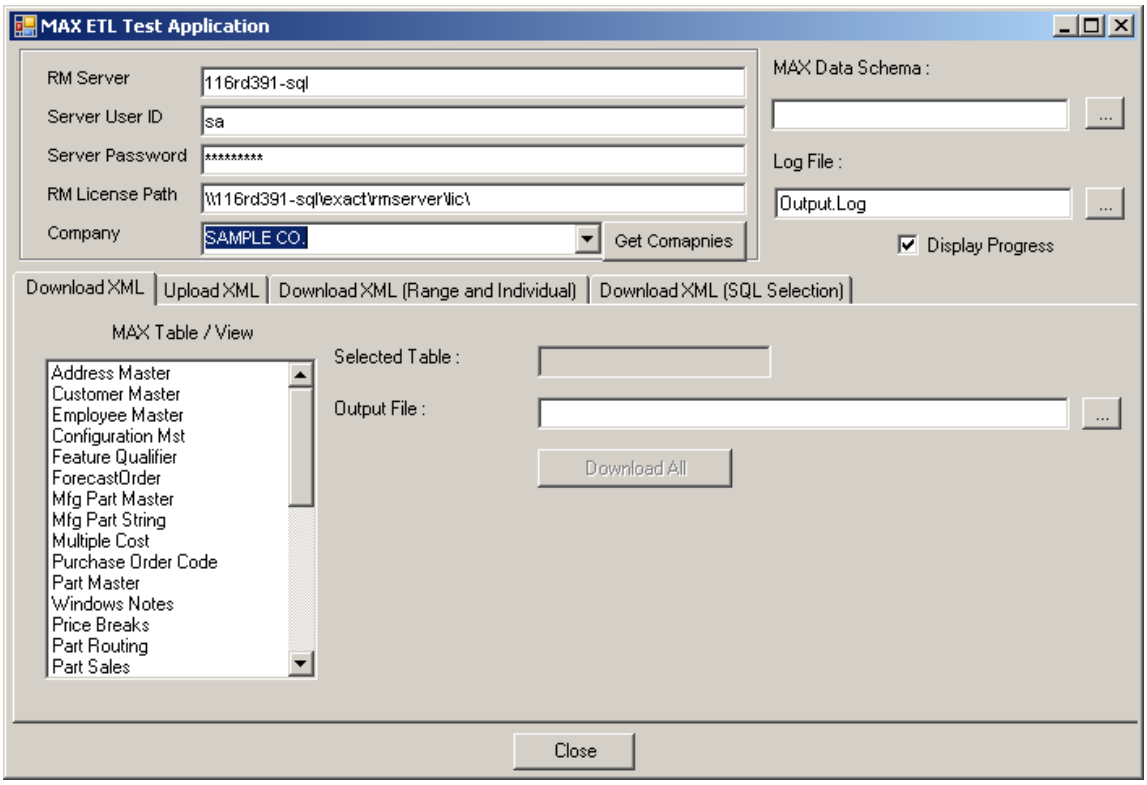


Figure 1. Download All Records

The VB.NET that gets executed when the “Download All” button is shown below.

```
Dim sConnString As String
If Not Checkfields() Then Return

' Check if the Selected Table Textbox has a vaule
Dim sTable As String = txt_Selected.Text
If sTable.Trim.Length = 0 Then
    MsgBox("No Table is selected")
    Return
End If

sConnString = "Provider=sqloledb;Data Source=" & txtServer.Text &
";database=ExactRM;UID=" & txtUID.Text & ";Pwd=" & txtPassword.Text &
";"
        MAXINTBO.Utils.SetConnectionString(cmbCompanies.Text,
txtLicensePath.Text, Application.StartupPath, sConnString)

Dim objExtract As MAXINTBO.DataExtractor = New MAXINTBO.DataExtractor



objExtract.MAXXMLSchema = txt_Schema.Text
' Set MAX  Schema Path
objExtract.LogFile = txt_LogFile.Text
' Set Log file Status
objExtract.MAXTable = txt_Selected.Text
' Set MAX  Table to extract Data
```

```
objExtract.XMLOutputFile = txt_OutputFile.Text
' Set XML Output File
'If the user wants to hide the Status dialog set Silent to TRUE
If chk_DispStatus.Checked Then objExtract.Silent = False Else
objExtract.Silent = True
objExtract.ExtractXML()
```

First MAXINTBO is set with the connection string information. Next, an instance of the DataExtractor object is created from MAXINTBO (Reference created from MAXINTBO.DLL). The object is initialized with MAX XML Schema, Log file, MAX Table name (The Table Names are described in the Table 10.4) and Output file. If the status dialog should be hidden the “Silent” property of the object should be set to True (The “Silent” property is false by default). After the above properties are set the ExtractXML() function is called to download MAX data into the XML document.

2. Upload the XML Data from a XML file to MAX Data.

Figure 2. shows the dialog where the user selects a file to be uploaded. The Input XML file is entered in the Text box or browsed by clicking the "…" button. Clicking the "Upload File" button uploads the contents of the input XML file into the MAX Data Table.

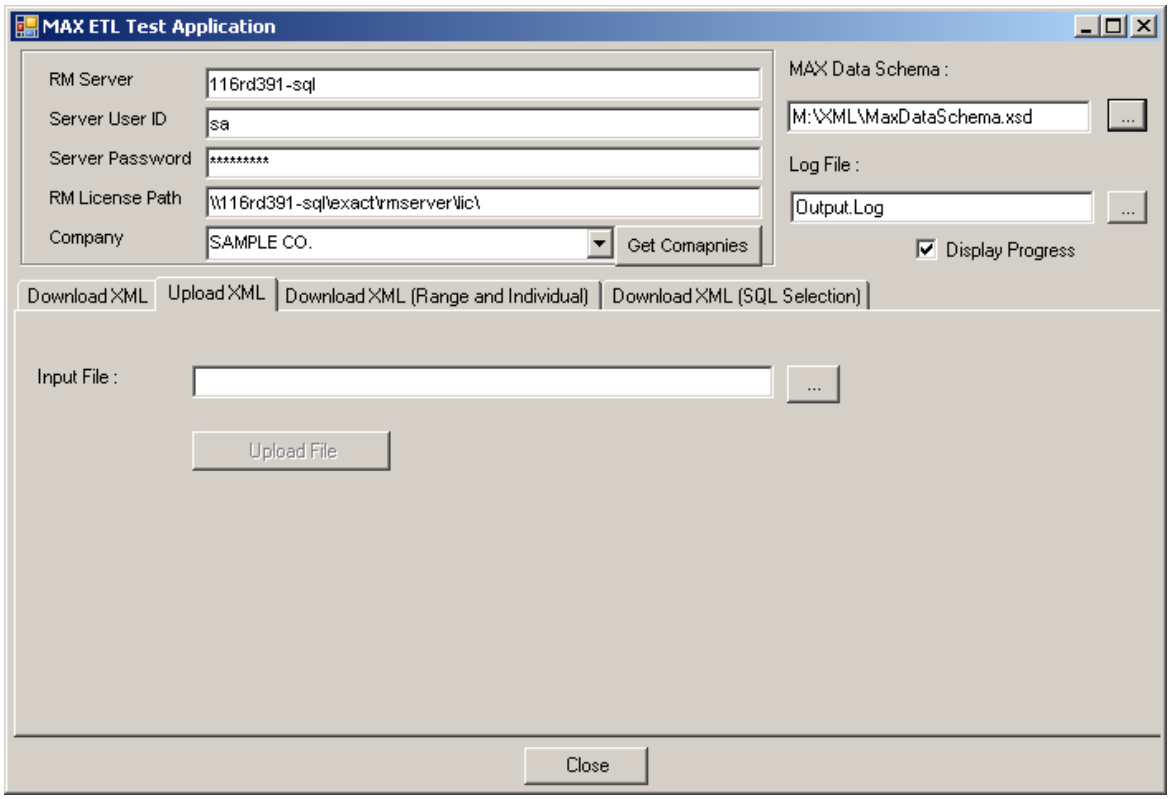


Figure 2. Upload XML file.

The VB.NET that gets executed when the "Upload File" button is shown below.

```
Dim sConnString As String

If Not Checkfields() Then Return
sConnString = "Provider=sqloledb;Data Source=" & txtServer.Text &
";database=ExactRM;UID=" & txtUID.Text & ";Pwd=" & txtPassword.Text &
";"
        MAXINTBO.Utils.SetConnectionString(cmbCompanies.Text,
txtLicensePath.Text, Application.StartupPath, sConnString)

Dim objUpload As New MAXINTBO.DataUploader

objUpload.MAXXMLSchema = txt_Schema.Text
' Set MAX  Schema Path
objUpload.XMLInputFile = txt_InputFile.Text
' Set XML Input File
objUpload.LogFile = txt_LogFile.Text
' Set Log file Status

'If the user wants to hide the Status dialog set Silent to TRUE
If chk_DispStatus.Checked Then objUpload.Silent = False Else
objUpload.Silent = True
```

```
objUpload.UploadXMLDocument()
```

First `MAXINTBO is set with the connection string information`. Next an instance of the DataUploader object is created from MAXINTBO (Reference created from MAXINTBO.DLL). The object is initialized with MAX XML Schema, Output file and Log file. If the status dialog should be hidden the "Silent" property of the object should be set to True (The "Silent" property is false by default). After the above properties are set the UploadXMLDocument() function is called to upload the XML document.

3. Download the Records from "Part Master" Table into an XML File based on a range or a list of Parts

Figure 3. shows the dialog where the user selects to download a range or list of parts from the "Part Master" Table. The Output file is PartMaster.xml. Clicking the "Download Range / Individual" button downloads all the records from the "Part Master" Table that match the Range or Individual options.

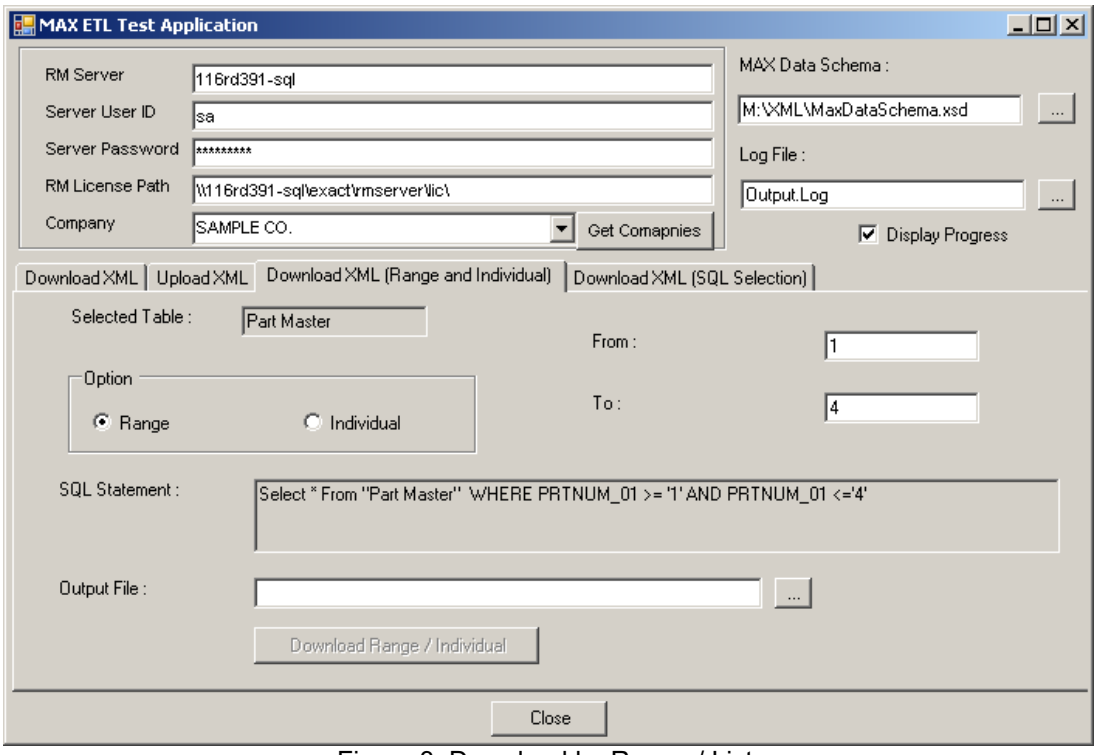


Figure 3. Download by Range / List.

The VB.NET that gets executed when the "Download Range / Individual" button is shown below.

```
Dim sConnString As String
If Not Checkfields() Then Return

' Check if the Selected Table Textbox has a value
Dim sTable As String = txt_SelTable.Text
If sTable.Trim.Length = 0 Then
      MsgBox("No Table is selected")
      Return
End If
```

```
sConnString = "Provider=sqloledb;Data Source=" & txtServer.Text &
";database=ExactRM;UID=" & txtUID.Text & ";Pwd=" & txtPassword.Text &
";"
        MAXINTBO.Utils.SetConnectionString(cmbCompanies.Text,
txtLicensePath.Text, Application.StartupPath, sConnString)

Dim objExtract As MAXINTBO.DataExtractor = New MAXINTBO.DataExtractor


objExtract.MAXXMLSchema = txt_Schema.Text
' Set MAX  Schema Path
objExtract.LogFile = txt_LogFile.Text
' Set Log file Status
objExtract.MAXTable = sTable
' Set MAX  Table to extract Data
objExtract.XMLOutputFile = txt_OutputFile2.Text
' Set XML Output File

'If the user wants to hide the Status dialog set Silent to TRUE
If chk_DispStatus.Checked Then objExtract.Silent = False Else
objExtract.Silent = True

'Set the WHERE Clause
objExtract.WhereClause = m_sWhere

objExtract.ExtractXML()
```

First MAXINTBO is set with the connection string information. Next, an instance of the DataExtractor object is created from MAXINTBO (Reference created from MAXINTBO.DLL). The object is initialized with MAX XML Schema, Log file, MAX Table name (The Table Names are described in the Table 10.4) and Output file. If the status dialog should be hidden, the “Silent” property of the object should be set to True (The “Silent” property is false by default). The WHERE clause (as displayed in the “SQL Statement” Text Box) is set to the WhereClause property of the Extract object. After the above properties are set the ExtractXML() function is called to download MAX data into the XML document.

4. Download the Records from a selected MAX Table based upon the where clause entered by the user.

Figure 4. shows the dialog where the user selects the MAX Table, browses and chooses the Output file. A customized WHERE clause is entered in the “WHERE Clause” edit box. Clicking the “Download SQL Selection” button downloads all the records from the “Part Master” Table that match the WHERE clause.

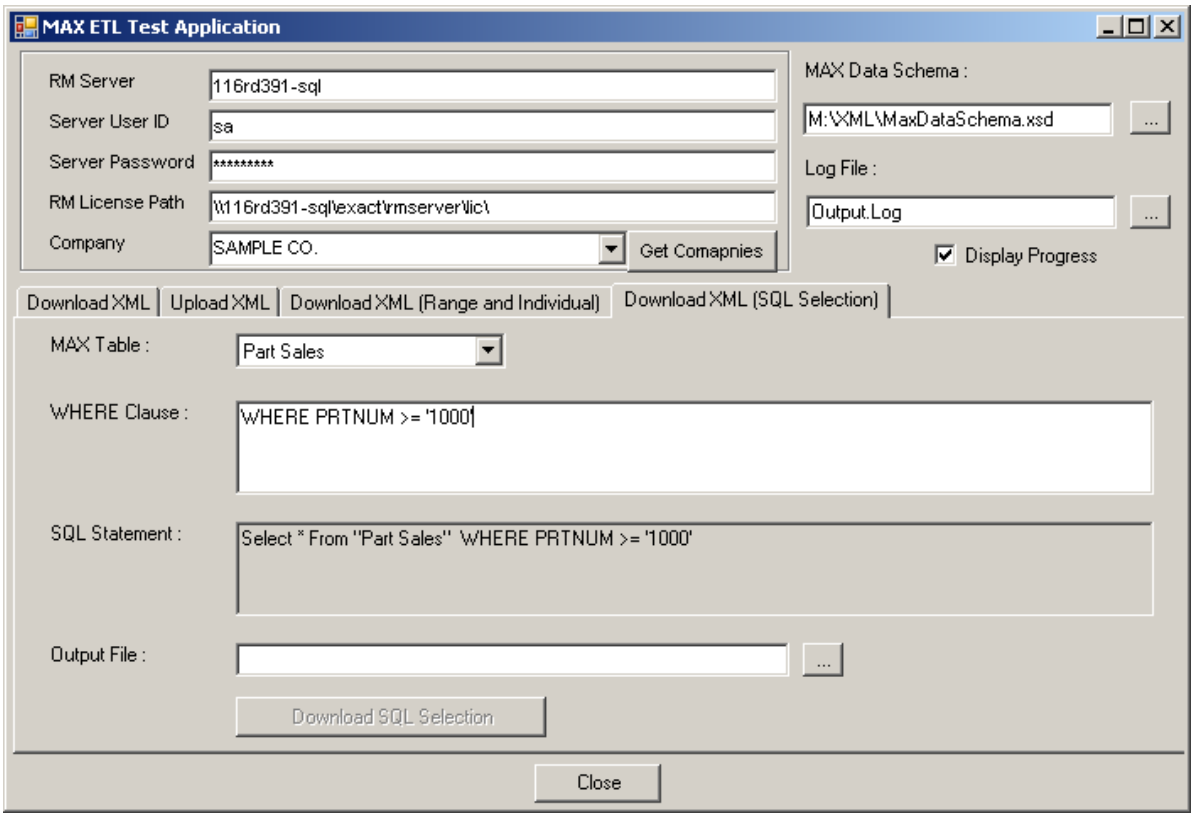


Figure 4. Download Records based on a WHERE clause.

The VB.NET that gets executed when the “Download SQL Selection” button is shown below.

```
Dim sConnString As String

If Not Checkfields() Then Return

' Check if the Selected Table Textbox has a vaule
Dim sTable As String = cbo_Table.Text
If sTable.Trim.Length = 0 Then
      MsgBox("No Table is selected")
      Return
End If

sConnString = "Provider=sqloledb;Data Source=" & txtServer.Text &
";database=ExactRM;UID=" & txtUID.Text & ";Pwd=" & txtPassword.Text &
";"
        MAXINTBO.Utils.SetConnectionString(cmbCompanies.Text,
txtLicensePath.Text, Application.StartupPath, sConnString)

Dim objExtract As MAXINTBO.DataExtractor = New MAXINTBO.DataExtractor

objExtract.MAXXMLSchema = txt_Schema.Text
' Set MAX  Schema Path
objExtract.LogFile = txt_LogFile.Text
' Set Log file Status
objExtract.MAXTable = sTable
' Set MAX  Table to extract Data
objExtract.XMLOutputFile = txt_OutputFile3.Text
```

```
' Set XML Output File

'If the user wants to hide the Status dialog set Silent to TRUE
If chk_DispStatus.Checked Then objExtract.Silent = False Else
objExtract.Silent = True

'Set the WHERE Clause
objExtract.WhereClause = txt_WhereClause.Text

objExtract.ExtractXML()
```

First, MAXINTBO is set with the connection string information. Next, an instance of the DataExtractor object is created from MAXINTBO (Reference created from MAXINTBO.DLL). The object is initialized with MAX XML Schema, Log file, MAX Table name (The Table Names are described in the Table 10.4) and Output file. If the status dialog should be hidden the “Silent” property of the object should be set to True (The “Silent” property is false by default). The WHERE clause as displayed in the “SQL Statement” Text Box is set to the WhereClause property of the Extract object. After the above properties are set, the ExtractXML() function is called to download MAX data into the XML document.

10.3 Writing your Extract\Load application

To create a new application that utilizes ETL functionality with MAX Update.NET:

1. Create a new VB.NET /C# Winforms project.
2. Add a reference to MAXINTBO.DLL from the “Solutions Explorer” tab by browsing and choosing the MAXINTBO.DLL from the EFW folder.
   After adding this reference, the developer can access the MAXINTBO class library.
3. Refer to the examples in the previous section on how to use the DataExtractor and DataUploader object.

10.4 MAX Business Objects, XML Elements and Table Names

Table 10.4.1 lists the MAX Business Objects and their respective XML Element Names and Table Names that should be used while setting the name of the Table in the Data Extractor Object.

Business Object	**XML Element**	**MAX Table/View Name**
Address Master	<Address_Master_Table>	Address Master
Customer Master	<Customer_Master_Table>	Customer Master
Employee Master	<Employee_Master_Table>	Employee Master
Feature/Option Descriptions	<Configuration_Mst_Table>	Configuration Mst
Feature Qualifier	<Feature_Qualifier_Table>	Feature Qualifier
Forecast Order	<ForecastOrder_Table>	ForecastOrder
MPN	<Mfg_Part_Master_Table>	Mfg Part Master
MPN String	<Mfg_Part_String_Table>	Mfg Part String
Multiple Cost	<Multiple_Cost_Table>	Multiple Cost
Non-Inventory PO	<Purchase_Order_Code_Table>	Purchase Order Code
Part Master	<Part_Master_Table>	Part Master
Part Notes	<Windows_Notes_Table>	Windows Notes
Price Breaks	<Price_Breaks_Table>	Price Breaks
Part Routing	<Part_Routing_Table>	Part Routing
Part Sales	<Part_Sales_Table>	Part Sales
Part Vendor	<Part_Vendor_Table>	Part Vendor
Product Schedule	<Product_Schedule_Table>	Product Schedule
Product Structure (Bill of Materials)	<Product_Structure_Table>	Product Structure
Purchase Order Code (Purchase Order)	<Purchase_Order_Code_Table>	Purchase Order Code
Purchase Order Note	<Purchase_Order_Note_Table>	Purchase Order Note
Purchase Requisition	<PurchaseRequisition_Table>	PurchaseRequisition
RMA Detail	<RMA_Detail_Table>	RMA Detail
Routing Notes	<Routing_Notes_Table>	Routing Notes
Sales Order	<SO_Master_Table>	SO Master
Sales Order Shipment	<SalesOrderShipment_Table>	SalesOrderShipment
Scheduled Price	<Scheduled_Price_Table>	Scheduled Price
Shipping Master	<Shipping_Master_Table>	Shipping Master
Ship History	<Ship_History_Table>	Ship History
Shop Order	<ShopOrder_Table>	ShopOrder
Stock Master	<Stock_Master_Table>	Stock Master
Transaction History	<Transaction_History_Table>	Transaction History
Vendor Master	<Vendor_Master_Table>	Vendor Master
WIP Data	<WIP_Data_Table>	Not Applicable
Work Center	<SFC_Work_Center_Table>	SFC Work Center

Table 10.4.1: MAX Business Objects

11. Support for ASP Programming

How to make use of MaxUpdateXML in ASP pages:

Environment: The MAX database and Internet Information Server (IIS) and MAXExact are supported only when all are installed on the same Windows 2000 / 2003 server, per the MAX Gold Standard. If IIS and MAX database are on different drives of that same machine then a virtual directory for the MAX Data Drive must be created in IIS. Directory security must be set according to the customer's company policy.

Microsoft Development Tools: Visual Studio. ASP

Language used in sample: VB Script

From ASP page, first create the object and immediately after creation initialize the object by calling InitASPXMLWrapper function as shown below:

```
Set ASPMaxUpdateWrapper = Server.CreateObject ("MaxUpdateXML.ASPXMLWrapper")
ASPMaxUpdateWrapper.InitASPXMLWrapper(ByVal PriConnectionString As String, _
ByVal CompanyName As String, _
ByVal LicensePath As String, _
ByVal LogPath As String, _
Optional ByVal bErrRpt As Boolean = False)
```

Notes:

PriConnectionString The primary database connection string. This connection string points to the ExactRM database.

CompanyName is the MAX Company Name.

LicensePath is the path to the ExactRM.lic file.

sLogPath is the directory relative to server where errors in your ASP call will be logged in a file named ASPxmlwrapper.log. (Log path format may be like “C:\Exact\MAX \Log1” or “D:\Exact\MAX \Log2” etc. Make sure the directory has necessary rights for ASP user to create and write a file.)

bErrRpt is a flag to set if you wish for an error log to be generated.

Example:

```
ASPMaxUpdateWrapper .InitASPXMLWrapper ("Provider=sqloledb;Data
Source=server1;database=ExactMAX;UID=sa;Pwd=xxxxx", “Sample Company”,
“C:\EXACT\RMServer\LIC”, “c:\logs”, false)
```

Now other function calls (In ASPXMLWrapper) can be made.

Even though you can test functions like

```
ASPMaxUpdateWrapper.ProcessTransXML("")
```

by passing “” or invalid xml, try to pass valid xml, because, in a worst case scenario, MAX MFC dlls (maxordr2.dll/maxtran2.dll/maxexec2.dll) on the server might try to open an MFC error message box which will cause your asp client to hang. Most errors caused by invalid XML have been resolved, but we recommend attempting to pass only valid XML.

Most of the error messages will be recorded to ASPxmlwrapper.log.

Note: This functionality is ready for use in a development and test environment only. Transaction functions, such as stock transfers and cycle counts should not trigger error messages, and neither should basic sales order header and line item order functions. More complex conditions that trigger error dialogs could cause an MAX ASP application to hang.

11.1 Setting up the ASP.NET environment

The biggest difference between using MAXUpdate in an ASP environment and as a standalone EXE is the additional security setup required to allow it to run. First, log in to the console of the server as the user which the ASP application is running as and verify that the user has sufficient rights to launch and use the MAX client. For the ASP.NET environment you will need to set up a user whit the rights listed below, the Application Pool for your website will need to be set up to run as this user.

11.1.1 File Permissions:

The user needs read rights to the following folders (SysWOW64 is for 64-bit OSs only):

- \\MAXServer\Exact_Shared\Exact\RMServer\LIC
- C:\Windows\SysWOW64\config\systemprofile\AppData
- C:\Windows\System32\config\systemprofile\AppData

The user also needs read/write rights to the following folders:

- \\MAXServer\Exact_Shared\Exact\RMServer\LOG
- C:\Exact\RMClient\EFW

11.1.2 SQL Server Permissions:

The user needs the VIEW SERVER STATE right on the SQL Server as well as db_datareader and db_datawriter roles on the ExactMAX and company databases.

11.1.3 DLLs

The ERMRemCl.DLL and ExactRMEnc.DLL need to be copied to the appropriate location where the “executable” will be running from. In the ASP.NET environment this is the C:\Windows\System32\inetsrv folder, if your server is running a 64-bit OS they need to be copied to C:\Windows\SysWOW64\inetsrv instead.

11.1.4 IIS Application Pool

The Application Pool for the webpage that will be using MAXUpdate needs to run as a specific user as mentioned above, also it needs to be properly configured to use .NET Framework Version 2.0 as well as being set to Enable 32-Bit Application (if you are running a 64-bit OS).

12. Function Descriptions

The following is a description of the various calls contained in the MAXUpdate Libraries. The functions are defined using the C Language syntax. For those not familiar with C, a glossary is provided at the end of this document that explains the terms. Please note that these functions are no longer recommended for utilizing MAXUpdate, they are included for reference for customers still needing to maintain older code. It is recommended that new applications use the calls to the MAXUpdateXML.DLL listed earlier in the documentation.

MAXUpdate users must use the wrapper functions described in the function descriptions.

Note: For 32-bit Applications:

The functions that require a pointer to an input record (such as AddSalesOder or ProcessTrans in MAXTRAN2.DLL) assume that the input record has a 4 byte structure alignment.

In Visual Basic the structures (custom types) have this alignment and the alignment can not be changed.

For C/C++ users, the header files have structures set to the correct alignment. If you make your own versions of these structures, ensure that the structure alignment is set to 4 bytes.

12.1 MAXORDER2 DLL

12.1.1 Initialization

- **OBJHANDLE DLLEXPORT InitMAXOrder (LPSTR PriConnectionString, LPSTR CompanyName, LPSTR LicensePath, LPSTR LogPath, BOOL bIsASP = FALSE, BOOL ErrRpt = FALSE);**

Parameter	Description
PriConnectionString	The primary database connection string. This connection string points to the ExactMAX database.
CompanyName	The MAX company that will have data inserted or updated.
LicensePath	Path to the ExactRM.lic file.
LogPath	Path to where the log file will be written to.
ErrRpt	Flag to specify if an error report should be generated.

Returns
A handle (32-bit pointer) to the created object. This handle must be used on subsequent calls to the DLL.

Comments
If the PriConnectionString, CompanyName, LicensePath are left blank, the dll will use the settings stored in the registry to fill in these three parameters.

This call must be made before using any of the other MAX Order2 DLL calls. It does all the setup necessary to work with MAX Orders.

- **int DLLEXPORT ShutdownMAXOrder (OBJHANDLE MaxOrderObject);**

Parameter	Description
MaxOrderObject	The OBJHANDLE obtained from the InitMAXOrder call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Call when done using the MAX Order2 DLL. Performs all necessary cleanup.

- **int DLLEXPORT SetVisualErrorReporting (OBJHANDLE MaxTransObject, BOOL ErrorReporting);**

Parameter	Description
MaxOrderObject	The OBJHANDLE obtained from the InitMAXTrans call.
ErrorReporting	Enable or disable visual error reporting (See above.)

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Use this call to enable or disable visual error reporting "on the fly".

12.1.2 Forecast Orders

- **int DLLEXPORT AddForecastOrder (OBJHANDLE Obj, LPSTR PartID, LPSTR CurDue, double CurQty, LPSTR OrdRef);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PartID	MAX Part Number.
CurDue	Current Due Date (YYMMDD)
CurQty	Current quantity.
OrdRef	Order Reference string.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Forecast Order. Requires CurQty argument to be given in Sales unit of measure

- **int DLLEXPORT ChangeForecastOrder (OBJHANDLE Obj, LPSTR OrdNum, char Status, LPSTR CurDue, double CurQty, LPSTR OrdRef);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdNum	MAX Order Number.
Status	Status of the Forecast Order
CurDue	Current Due Date (YYMMDD)
CurQty	Current quantity.
OrdRef	Order Reference string.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Forecast Order.

- **int DLLEXPORT DeleteForecastOrder (OBJHANDLE Obj, LPSTR OrdNum);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdNum	MAX Order Number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Forecast Order.

12.1.3 Sales Orders

- **int DLLEXPORT AddSalesOrder (OBJHANDLE Obj, void far *SoeMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeMstRecord	Address of an MAX SoeMst Record (See header file *SoeMst.h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Adds an MAX Sales Order, using the next available order number.

- **int DLLEXPORT ChangeSalesOrder (OBJHANDLE Obj, void far *SoeMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeMstRecord	Address of an MAX SoeMst Record (See header file *SoeMst.h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an MAX Sales Order header.

- **int DLLEXPORT DeleteSalesOrder (OBJHANDLE Obj, LPSTR OrdNum);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdNum	MAX Order Number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an MAX Sales Order, including all line items and notes.

- **int DLLEXPORT AddSalesOrderLineItem (OBJHANDLE Obj, void far *SoeDetRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeDetRecord	Address of an MAX SoeDet Record (See header file *SoeDet.h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Sales Order Line Item.

- **int DLLEXPORT DeleteSalesOrderLineItem (OBJHANDLE Obj, void far *SoeDetRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeDetRecord	Address of an MAX SoeDet Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Sales Order Line Item.

- **int DLLEXPORT ChangeSalesOrderLineItem (OBJHANDLE Obj, void far *SoeDetRecord, void far *OldSoeDetRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeDetRecord	Address of an MAX SoeDet Record
OldSoeDetRecord	Address of the existing SoeDet Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Sales Order Line Item.

- **int DLLEXPORT AddSOLineItem (OBJHANDLE Obj, void far *SoeDetRecord, BOOL bSuppressMsg = FALSE);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
SoeDetRecord	Address of an MAX SoeDet Record
bSuppressMsg	If set to TRUE, message boxes for errors will be suppressed.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
When adding a feature option configured sales order into MAX, a Windows message box displays to notify the user that an MS order has been added. When adding orders in an unattended mode via MAXUpdate, the pop up message box stops the automation. ***To suppress the Planned Order creation message box, set the bSuppressMsg parameter to True. (False is the default).***

- **double DLLEXPORT GetUnitPrice (OBJHANDLE Obj, LPSTR PartNumber, double Qty, LPSTR CustID = NULL);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PartNumber	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID (optional)

Returns
a double (the unit price).

Comments
This is the price with any applicable discounts.

- **double DLLEXPORT GetStandardPrice (OBJHANDLE Obj, LPSTR PartNumber, double Qty, LPSTR CustID = NULL);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PartNumber	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID (optional)

Returns
a double (the standard price).

Comments
This is the price with no discounts.

- **double DLLEXPORT GetDiscount (OBJHANDLE Obj, LPSTR PartNumber, double Qty, LPSTR CustID);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PartNumber	MAX Part Number
Qty	Quantity to be purchased
CustID	MAX Customer ID

Returns
a double.

Comments
The value returned is the discount percentage, **(**e**.**g. 20 for a 20% discount).

- **double DLLEXPORT ConvertQuote (OBJHANDLE Obj, LPSTR OrderNumber, BOOL bSuppressMsg = FALSE);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrderNumber	Quote number
bSuppressMsg	If set to TRUE, message boxes for errors will be suppressed.

Returns
a long

Comments
The value returned is 0 in case of an error or the number of the newly created Sales Order.

12.1.4 Purchase Orders

- **int DLLEXPORT ChangePOHeading (OBJHANDLE Obj, void far * POCodeRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
POCodeRecord	Address of an MAX POCode Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order header.

- **int DLLEXPORT DeletePurchaseOrder (OBJHANDLE Obj, LPSTR OrdNum);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdNum	MAX Order number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Purchase Order heading, the associated line items and notes.

- **int DLLEXPORT AddPurchaseOrderLineItem (OBJHANDLE Obj, void far *OrdMstRecord, BOOL IncOrdRev, BOOL CreateHeader);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record (See header file *OrdMst.h).*
IncOrdRev	If TRUE, increments the PO revision number
CreateHeader	If TRUE, creates a PO header record in *POCode.dat*.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Purchase Order Line Item.

- **int DLLEXPORT DeletePurchaseOrderLineItem (OBJHANDLE Obj, void far *OrdMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Purchase Order Line Item.

- **int DLLEXPORT ChangePurchaseOrderLineItem (OBJHANDLE Obj, void far *OrdMstRecord, BOOL IncOrdRev);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record
IncOrdRev	If TRUE, increments the PO revision number

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order Line Item.

- **int DLLEXPORT ChangePOInTransit (OBJHANDLE Obj, void far *OrdMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Order Line Item which is In Transit.

- **int DLLEXPORT AddNonInventoryPOLineItem (OBJHANDLE Obj, void far * OrdmstRecord, BOOL IncOrdRev, BOOL CreateHeader);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX OrdMst Record Record (See header file *OrdMst.h*).
IncOrdRev	If TRUE, increments the PO revision number
CreateHeader	If TRUE, creates a PO header record in *POCode.dat*.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Non-Inventory Purchase Order Line Item.

- **int DLLEXPORT DeleteNonInventoryPOLineItem (OBJHANDLE Obj, void far * OrdmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX OrdMst Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Non-Inventory Purchase Order Line Item.

- **int DLLEXPORT ChangeNonInventoryPOLineItem (OBJHANDLE Obj, void far * OrdmstRecord, BOOL IncOrdRev);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX OrdMst Record
IncOrdRev	If TRUE, increments the PO revision number

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Order Line Item.

- **int DLLEXPORT ChangeNonInventoryPOInTransit (OBJHANDLE Obj, void far *OrdmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX OrdMst Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Order Line Item that is In Transit.

12.1.5 Purchase Requisitions

- **int DLLEXPORT AddPurchaseRequisition (OBJHANDLE Obj, void far *OrdMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record (See header file *OrdMst.h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Purchase Requisition.

- **int DLLEXPORT DeletePurchaseRequisition (OBJHANDLE Obj, void far *OrdMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Purchase Requisition.

- **int DLLEXPORT ChangePurchaseRequisition (OBJHANDLE Obj, void far *OrdMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdMstRecord	Address of an MAX OrdMst Record

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Purchase Requisition.

- **int DLLEXPORT AssignPONumber (OBJHANDLE Obj, LPINT PRsAssigned, LPINT POsCreated, LPLONG LastPO, BOOL CreatePartVendor, LPSTR ApprovedBy)**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PRsAssigned	Used to return the number of Purchase Requisitions assigned.
POsCreated	Used to return the number of Purchase Orders that are created.
LastPO	Returns the last PO number created.
CreatePartVendor	If True will automatically create a new Part Vendor relationship and process the order. If FALSE and there is not an existing Part Vendor relationship, the order will be skipped.
ApprovedBy	If a value is passed in for Approved By, only the orders that have a matching APRDBY value will be processed.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Assigns PO Numbers to Planned Orders and Purchase Requisitions with a valid Vendor ID and a Status of 1, 2 or 3.

- **int DLLEXPORT ReleaseOrders (OBJHANDLE Obj, LPSTR StartPart, LPSTR EndPart, LPSTR StartBuyer, LPSTR EndBuyer, LPSTR StartOrder, LPSTR EndOrder)**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
StartPart	Used to set the first part of a range. Set to blank for all parts within the order range.
EndPart	Used to set the last part of a range. Set to blank for all parts within the order range.
StartBuyer	Used to set the first buyer of a range. Set to blank for all buyers within the order range.
EndBuyer	Used to set the last buyer of a range. Set to blank for all buyers within the order range.
StartOrder	Set starting order for range.
EndOrder	Set ending order for range.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Converts approved Planned Orders into Purchase Requisitions. This routine performs the same processing as Print Purchase Requisitions.

- **int DLLEXPORT AddNonInventoryPurchaseRequisition(OBJHANDLE Obj, void far *OrdmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX PONoteA Record (See header file *OrdMst.h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Non-Inventory Purchase Requisition.

- **int DLLEXPORT DeleteNonInventoryPurchaseRequisition (OBJHANDLE Obj, void far *OrdmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
OrdmstRecord	Address of an MAX Order Master Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Non-Inventory Purchase Requisition.

- **int DLLEXPORT ChangeNonInventoryPurchaseRequisition (OBJHANDLE Obj, void far * OrdmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder
OrdmstRecord	Address of an MAX Order Master Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Non-Inventory Purchase Requisition.

- **int DLLEXPORT AssignNonInvPONumber(OBJHANDLE Obj, LPINT PRsAssigned, LPINT POsCreated, LPLONG LastPO, LPSTR ApprovedBy);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PRsAssigned	Used to return the number of Purchase Requisitions assigned.
PosCreated	Used to return the number of Purchase Orders that are created.
LastPO	Returns the last PO number created.
ApprovedBy	If a value is passed in for Approved By, only the orders that have a matching APPROV value will be processed.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Assigns PO Numbers to Purchase Requisitions with a valid Vendor ID.

12.1.6 Shop Orders

- **int DLLEXPORT AddUpdateShopOrder (void far *OrdmstRecord, BOOL bBlowThruPseudos, BOOL bXMLfields, LPSTR sError);**

Parameter	Description
OrdmstRecord	Address of an MAX Ordmst Record (See header file *Ordmst.h*).
Blow Through Pseudos/Phantoms	1 if Pseudos and Phantoms should be blown through when creating the default requirement, 0 otherwise.
bXML Fields	False unless called by MAX ETL.
sError	Returns the text of the error message.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Either adds or updates an MAX Shop Order. If the order number is left blank an order will be added using the next available order number. The order will also be added if the order number is not blank and the order does not currently exist. Either MS or MF orders can be added (just set the TYPE_10 field to the appropriate type in the Ordmst record). Both the default requirements and default routings will be created.

12.1.7 Vendor Master

- **int DLLEXPORT AddVendor(OBJHANDLE Obj, void far *VenMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
VenMstRecord	Address of an MAX VenMst Record (See header file Venmst.*h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Vendor.

- **int DLLEXPORT DeleteVendor (OBJHANDLE Obj, LPSTR Venid);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
VenMstRecord	Address of an MAX VenMst Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Vendor.

- **int DLLEXPORT ChangeVendor (OBJHANDLE Obj, void far * VenmstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
VenMstRecord	Address of an MAX VenMst Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Vendor.

12.1.8 Part Vendor

- **int DLLEXPORT AddPartVendor(OBJHANDLE Obj, void far *PrtVenRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PrtVenRecord	Address of an MAX PrtVen Record (See header file PrtVen.*h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Part Vendor relationship.

- **int DLLEXPORT DeletePartVendor (OBJHANDLE Obj, LPSTR Venid, LPSTR Prtnum, LPSTR MfgPart, LPSTR MpnMfg);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
Venid	MAX Vendor.
Prtnum	MAX Part ID.
MfgPart	MAX Manufacturing Part Number.
MpnMfg	MPN Manufacturer

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Part Vendor relationship.

- **int DLLEXPORT ChangePartVendor (OBJHANDLE Obj, void far * PrtVenRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PrtVenRecord	Address of an MAX PrtVen Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Part Vendor record.

12.1.9 Purchase Part Data

- **int DLLEXPORT ChangePurchasePartData (OBJHANDLE Obj, void far *PrtMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
PrtMstRecord	Address of an MAX Part Master Record (See header file PrtMst.*h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes purchase part data for an existing MAX Part. Only the following fields can be changed: Purchasing planning lead time, buy lead time, stock lead time, buyer and recommended vendor can be changed.

12.1.10 Tax Codes

- **int DLLEXPORT AddTaxCode(OBJHANDLE Obj, void far *TaxMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
TaxMstRecord	Address of an MAX TaxMst Record (See header file TaxMst.*h*).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Creates a new MAX Tax code.

- **int DLLEXPORT DeleteTaxCode (OBJHANDLE Obj, void far *TaxMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
TaxMstRecord	Address of an MAX TaxMst Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Deletes an existing MAX Tax code.

- **int DLLEXPORT ChangeTaxCode (OBJHANDLE Obj, void far * TaxMstRecord);**

Parameter	Description
ObjHandle	Handle obtained from call to InitMAXOrder.
TaxMstRecord	Address of an MAX TaxMst Record.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Changes an existing MAX Tax code

12.1.11 Generic Record Interface

The generic record interface is to be used in situations where the applications programmer is unable or does not desire to use the standard MAX record structures. The following functions provide a simple, yet powerful methodology for dealing with MAX records.

- **OBJHANDLE DLLEXPORT GetSOEMSTRecord ();**

Parameter	Description
None	None

Returns
An object handle to an MAX SOEMST record.

Comments
Call this function to have MAX Update allocate an MAX SOEMST record. Use the Get/SetField functions to fill out or examine the fields of the record. Use the record in MAX Order2 calls. When done using the record, call ReturnSOEMSTRecord to free memory used by the record.

- **void DLLEXPORT ReturnSOEMSTRecord (OBJHANDLE Obj);**

Parameter	Description
Obj	Object handle to a SOEMST record previously obtained from **GetSOEMSTRecord**

Returns
Nothing

Comments
Returns SOEMST record to the system, freeing associated memory.

- **OBJHANDLE DLLEXPORT GetSOEDETRecord ();**

Parameter	Description
None	None

Returns
An object handle to an MAX SOEDET record.

Comments
Call this function to have MAX Update allocate an MAX SOEDET record. Use the Get/SetField functions to fill out or examine the fields of the record. Use the record in MAX Order2 calls. When done using the record call ReturnSOEDETRecord to free memory used by the record.

- **void DLLEXPORT ReturnSOEDETRecord (OBJHANDLE Obj);**

Parameter	Description
Obj	Object handle to a SOEDET record previously obtained from **GetSOEDETRecord**

Returns
Nothing

Comments
Returns SOEDET record to the system, freeing associated memory.

- **int DLLEXPORT SetCharField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, char data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary)
data	Character to be placed in the selected field.

Returns
An integer error code. The possible values are as follows:

Error Name	Value	Description
MAXERR_SUCCESS	0	No error, OK
MAXERR_BADOBJECT	-1	Object pointer was bad
MAXERR_BADOFFSET	-2	Record Offset is out of range
MAXERR_BADDATATYPE	-3	Data types do not match
MAXERR_FIELDNOTFOUND	-4	Field not found in record
MAXERR_STRINGTOOLONG	-5	Length of string too long for field
MAXERR_BADDATASIZE	-6	Wrong data size for field

Note: Error codes are the same for all Get/SetField functions.

Comments
Used to set a character field in an MAX record.

- **char DLLEXPORT GetCharField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) e.*g., "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary)
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The character from the record field.

Comments
Used to get a character field from an MAX record.

- **int DLLEXPORT SetIntField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary)
	e.g. "SOEMST"
field	MAX field name (as defined in MAX Data Dictionary) *e.g. "SHPLBL"*
data	Integer to be placed in the selected field.

Returns
An integer error code. See error code table above.

Comments
Used to set an integer (2-byte) field in an MAX record.

- **int DLLEXPORT GetIntField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary)
	e.g. "SOEMST"
field	MAX field name (as defined in MAX Data Dictionary)
	e.g. :SHPLBL"
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The integer from the record field.

Comments
Used to get an integer field from an MAX record.

- **int DLLEXPORT SetLongField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, long data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary)
data	Long integer (4-bytes) to be placed in the selected field

Returns
An integer error code. See error code table above.

Comments
Used to set a long integer (4-byte) field in an MAX record.

- **long DLLEXPORT GetLongField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary)
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The long integer (4-bytes) from the record field.

Comments
Used to get a long integer field from an MAX record.

- **int DLLEXPORT SetFloatField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, float data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary) *e.g. "SPLIT1"*
data	Floating point number (4-bytes) to be placed in the selected field

Returns
An integer error code. See error code table above.

Comments
Used to set a floating point (4-byte) field in an MAX record.

- **float DLLEXPORT GetFloatField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary) e.g. *"COMMIS"*
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The floating point number (4-bytes) from the record field.

Comments
Used to get a floating point (4-byte) field from an MAX record.

- **int DLLEXPORT SetDoubleField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, double data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) e.g. *"SOEDET"*
field	MAX field name (as defined in MAX Data Dictionary) e.g. *"PRICE"*
data	Floating point number (8-bytes) to be placed in the selected field

Returns
An integer error code. See error code table above.

Comments
Used to set a floating point (8-byte) field in an MAX record.

- **double DLLEXPORT GetDoubleField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEDET"*
field	MAX field name (as defined in MAX Data Dictionary) *e.g. "CURQTY"*
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The floating point number (8-bytes) from the record field.

Comments
Used to get a floating point (8-byte) field from an MAX record.

- **int DLLEXPORT SetStringField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, LPSTR data);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEDET"*
field	MAX field name (as defined in MAX Data Dictionary) *e.g. "LINNUM"*
data	Address of string to be placed in the selected field

Returns
An integer error code. See error code table above.

Comments
Used to set a string field in an MAX record.

- **LPSTR DLLEXPORT GetStringField (OBJHANDLE Obj, LPSTR MaxFileNo, LPSTR field, int *retcode);**

Parameter	Description
Obj	Object handle to an MAX record previously obtained from a **Get**XXXXXX**Record** call
MaxFileNo	MAX File name (as defined in MAX Data Dictionary) *e.g. "SOEMST"*
field	MAX field name (as defined in MAX Data Dictionary) e.g. "STYPE"
retcode	Pointer (32-bit address) of an integer. Used to return the function error code. See error code table above.

Returns
The string from the record field.

Comments
Used to get a string field from an MAX record.

12.2 MAXTRAN2 DLL

12.2.1 Initialization

- **OBJHANDLE DLLEXPORT InitMAXTrans(LPSTR PriConnectionString, LPSTR CompanyName, LPSTR LicensePath, LPSTR LogPath, BOOL bIsASP = FALSE, BOOL ErrRpt = FALSE);**

Parameter	Description
PriConnectionString	The primary database connection string. This connection string points to the ExactRM database.
CompanyName	The MAX company that will have data inserted or updated.
LicensePath	Path to the ExactRM.lic file.
LogPath	Path to where the log file will be written to.
bIsASP	Flag to specify if the dll is being used in an ASP application.
ErrRpt	Flag to specify if an error report should be generated.

Returns
A handle (32-bit pointer) to the created object. This handle must be used on subsequent calls to the DLL.

Comments
If the PriConnectionString, CompanyName, LicensePath are left blank, the dll will use the settings stored in the registry to fill in these three parameters.
This call must be made before using any of the other MAX Transaction DLL calls. It does all the setup necessary to work with MAX Transactions.

- **int DLLEXPORT SetVisualError (OBJHANDLE MaxTransObject, BOOL ErrorReporting);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
ErrorReporting	Enable or disable visual error reporting (See above.)

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Use this call to enable or disable visual error reporting "on the fly".

12.2.2 Error Handling

- **int DLLEXPORT GetErrorCode (OBJHANDLE Obj);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is the error code set during the **ProcessTrans** call. If no error occurred, this value will be 0.

Comments
Use this call to obtain the error code if not using visual error reporting.

- **LPSTR DLLEXPORT GetErrorString (OBJHANDLE Obj);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
A far pointer to a zero-terminated string. The string is the formatted error string set during the ProcessTrans call. If no error occurred, the string will be empty.

Comments
Use this call to obtain the error string if not using visual error reporting.

- **int DLLEXPORT ShutdownMAXTrans (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Call when done using the MAX Order2 DLL. Performs all necessary cleanup.

12.2.3 Transaction Processing

- **int DLLEXPORT ProcessTrans (OBJHANDLE MaxTransObject, void far *TransBlock);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
TransBlock	Address of transaction structure (See *ITrans.h (C/C++) or ITrans.str (BASIC)* for Details).

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
Call to process MAX Transactions. Parameter information is passed in using the ITRANS structure (See ITrans.h and MAX Transaction Load documentation).

12.2.4 Serial Tracking Only Parts

For parts under serial control, but not lot control.

- **int DLLEXPORT CreateSerialArray (OBJHANDLE MaxTransObject, int TransactionQty);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
TransactionQty	The quantity for the transaction.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made before a ProcessTrans call for a transaction using Serial tracking with the Individual option and a quantity greater than 1.

- **int DLLEXPORT SetSerialNumber (OBJHANDLE MaxTransObject, int Index, LPCSTR SerialNumber);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the serial number. The index is 0 (zero) based, i.e. 0 is the number of the first index.
SerialNumber	The serial number.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each serial number that is to be created and must be done before the ProcessTrans call.

- **int DLLEXPORT DeleteSerialArray (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call should be made after a ProcessTrans call for a transaction using Serial tracking with the Individual option and a quantity greater than 1.

12.2.5 Lot Tracking Only Parts

For parts under lot control, but not serial control.

- **int DLLEXPORT CreateLotQtyArray (OBJHANDLE MaxTransObject, int LotCount);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
LotCount	The number of lots for this transaction.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made before a ProcessTrans call for a transaction using lot tracking if the transaction has more than one lot.

- **int DLLEXPORT SetLotQty (OBJHANDLE MaxTransObject, int Index, LPCSTR Lot, double Qty, long ExpirationDate, BOOL Qcode, LPCSTR Dispositon);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the lot number. The index is 0 (zero) based, i.e. 0 is the number of the first index.
LotNumber	The lot number.
Qty	The lot quantity.
ExpirationDate	The expiration date for the lot. It can be set to 0 if it is not used. The year is a four digit year and is stored in the high word. The month is stored in the high byte of the low word and the day is in the low byte of the low word.
Qcode	Quarantine flag (TRUE/FALSE)
Disposition	The lot disposition.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each lot number that is to be used and must be done before the **ProcessTrans** call.

- **int DLLEXPORT DeleteLotQtyArray (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call should be made after a ProcessTrans call for a transaction where a previous CreateLotQtyArray call has been made.

12.2.6 Lot and Serial Tracking Parts

- **int DLLEXPORT CreateLotAndSerialArray (OBJHANDLE MaxTransObject, int TransactionQty, int LotCount);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
TransactionQty	The quantity for the transaction.
LotCount	The number of lots for this transaction.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made before a ProcessTrans call for a transaction using lot and serial tracking. There is no specific Set function associated with this Create function, in the background it creates the separate Lot array and Serial array similar to the above calls for Lot only and Serial only parts. The functions SetSerialNumber and SetLotQty should be used as documented above to set the lot and serial data before calling ProcessTrans. To function correctly this should be done one lot at a time as there is no way to specify which lot a given serial number is associated with. This means that to transact an issue of 200 lot/serial parts with 50 from lot A and 150 from lot B, it should be split into two calls to ProcessTrans, once with quantity 50 and once with quantity 150. First you would call the CreateLotAndSerialArray function, then call SetLotQty, then SetSerialNumber fifty times, to set all 50 serial numbers, then the ProcessTrans Function and then DeleteLotAndSerialArray. After that is complete the process would be repeated for the lot with quantity 150. When using ProcessTransXML this is all taken care of automatically in the background by the logic in the XML function.

- **int DLLEXPORT DeleteLotAndSerialArray (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call should be made after a ProcessTrans call for a transaction where a previous CreateLotAndSerialArray call has been made.

12.2.7 Lot/Serial Control for Backflush and Post Production Completion

The following functions only apply to the **Backflush** and **Post Production Completion** transactions when the parent part is under lot and/or serial control or at least one component being processed is under lot and/or serial control. They should be used for these transactions instead of the functions above.

First a part array is created for all the parts under lot and/or serial control (including the parent and all components). Next a serial array must be created for each part under serial control and a lot array must be created for each part under lot control.

- **int DLLEXPORT CreateLotPartArray (OBJHANDLE MaxTransObject, int PartCount);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
PartCount	The number of parts under lot and or serial control. This includes the parent part and all components.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made before the ProcessTrans call.

- **int DLLEXPORT DeleteLotPartArray (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call should be made after a ProcessTrans call for a transaction where a previous CreateLotPartArray call has been made.

- **int DLLEXPORT SetLotPart (OBJHANDLE MaxTransObject, int Index, LPCSTR Part);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the lot number. The index is 0 (zero) based, i.e. 0 is the number of the first index.
Part	The part number under lot and/or serial control.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each part under lot and/or serial control before the ProcessTrans call.

- **int DLLEXPORT CreateSerialArrayForPart (OBJHANDLE MaxTransObject, int TransactionQty, int Part Index);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
TransactionQty	The serial quantity for this part.
PartIndex	The index for the part under serial control.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made for each part under serial control before the ProcessTrans call.

- **int DLLEXPORT SetSerialNumberForPart (OBJHANDLE MaxTransObject, int Index, LPCSTR SerialNumber, int PartIndex);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the serial number. The index is 0 (zero) based, i.e. 0 is the number of the first index.
SerialNumber	The serial number.
PartIndex	The index for the part under serial control.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each serial number that is to be created and must be done before the ProcessTrans call.

- **int DLLEXPORT CreateLotQtyArrayForPart (OBJHANDLE MaxTransObject, int LotCount, int PartIndex);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
LotCount	The number of lots for this part.
PartIndex	The index for the part under lot control.

Returns
An integer. Value is 1 if the serial array could be created, 0 on failure.

Comments
This call must be made before a ProcessTrans call for a transaction using lot tracking if the transaction has more than one lot.

- **int DLLEXPORT SetLotQtyForPart (OBJHANDLE MaxTransObject, int Index, LPCSTR Lot, double Qty, long ExpirationDate, BOOL Qcode, LPCSTR Disposition);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the lot number. The index is 0 (zero) based, i.e. 0 is the number of the first index.
LotNumber	The lot number.
Qty	The lot quantity.
ExpirationDate	The expiration date for the lot. It can be set to 0 if it is not used. The year is a four digit year and is stored in the high word. The month is stored in the high byte of the low word and the day is in the low byte of the low word.
Qcode	Quarantine flag (TRUE/FALSE)
Disposition	The lot disposition.
PartIndex	The index for the part under lot control.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each lot number that is to be used and must be done before the ProcessTrans call.

12.2.8 Stockroom selection for Backflush and Post Production Completion

The following functions for creating a stock room array and setting the stockroom apply only to **Backflush** and **Post Production Completion**. If a stock room array is not created, the issue of the component is done from its default stock room.

If you create a stock room array for a backflush transaction, you must set the stock room for each component (in ascending order of stock room) even if the default stock room is to be used.

- **int DLLEXPORT CreateStockRoomArray (OBJHANDLE MaxTransObject, int nComponentCount);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
nComponentCount	The number of components to be processed.

Returns
An integer. Value is 1 if the stock room array could be created, 0 on failure.

Comments
This call must be made before the ProcessTrans call.

- **int DLLEXPORT SetStockRoom (OBJHANDLE MaxTransObject, int Index, LPCSTR StockRoom, LPCSTR Part);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.
Index	The array index for the stock room . The index is 0 (zero) based, i.e. 0 is the number of the first index.
StockRoom	The stock room.
Part	The component part to be issued.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call must be made for each component that is to be issued and must be done before the ProcessTrans call.

- **int DLLEXPORT DeleteStockRoomArray (OBJHANDLE MaxTransObject);**

Parameter	Description
MaxTransObject	The OBJHANDLE obtained from the InitMAXTrans call.

Returns
An integer. Value is 1 on success, 0 on failure.

Comments
This call should be made after a ProcessTrans call for a transaction where a CreateStockRoomArray call has been made.

13. Database Changes

13.1 MAX 5.5.0 to MAX 5.5.1

File	Field	Old Type	New Type	Remark
Code_Master	DESC_36	char(20)	char(64)	Modify
Customer_Master	TYPE_23		char(159)	Add
ECO_Projects	Account Number	nvarchar(32)	nvarchar(159)	Modify
General_Ledger_Acct	GLACCT_19	char(32)	char(159)	Modify
Invoice_Detail	CLASS_32		char(159)	Add
Invoice_Detail	GLXREF_32	char(32)	char(159)	Modify
Invoice_Detail	WARRES_32		float	Add
Invoice_Master	GLXREF_31	char(32)	char(159)	Modify
Lot_Track_Structure	ISSCST_69		float	Add
Lot_Track_Structure	SORDNUM_69		char(12)	Add
Lot_Track_Structure	TNXID_69		bigint	Add
MAX_Transaction	CLASS_39		char(159)	Add
Order_Master	CLASS_10		char(159)	Add
Order_Master	JOB_10		char(159)	Add
Part_Sales	WARRES_29		float	Add
PO_Receipts	CLASS_55		char(159)	Add
Purchase_Order_Code	SHPVIA_16	char(20)	char(64)	Modify
Purchase_Order_Code	TERMS_16	char(20)	char(64)	Modify
Serial_Master	SORDNUM_71		char(12)	Add
Serial_Transaction	CLASS		char(159)	Add
SO_Detail	CLASS_28		char(159)	Add
SO_Detail	GLXREF_28	char(32)	char(159)	Modify
SO_Detail	JOB_28		char(159)	Add
SO_Detail	WARRES_28		float	Add
SO_Master	GLXREF_27	char(32)	char(159)	Modify
SubContract_Move	PMDES_101	char(25)	char(50)	Modify
Tax_Master	GLXREF_25	char(32)	char(159)	Modify
Transaction_History	CLASS_15		char(159)	Add
Vendor_Master	TYPE_08		char(159)	Add
Voucher	APACCT_56	char(32)	char(159)	Modify
Voucher	FRTACCT_56	char(32)	char(159)	Modify
Work_General_Ledger	ACCOUNT_58	char(32)	char(159)	Modify
Work_General_Ledger	CLASS_58		char(159)	Add

15. Glossary

DLL

Dynamic **L**ink **L**ibrary. A Windows library which resides in memory and can be called from multiple client applications. Loading and Unloading is handled automatically by Windows.

DLLEXPORT

Defined as **FAR PASCAL**. This means that the DLL function must be called far (32-bit address call) and the parameters are passed in the Pascal language fashion (parameters pushed on the stack from left to right, called function does stack cleanup).

DOUBLE

A **double** is an 8-byte floating point number.

LPCSTR

Defined as a const char far *. A 32-bit pointer to an ASCIIZ (null-terminated) string which can not be changed.

LPINT

Defined as an int far*. A 32-bit pointer to an integer (2 Byte).

LPLONG

Defined as a long far*. A 32-bit pointer to a long (4 Byte) integer.

LPSTR

Defined as a char far *. A 32-bit pointer to an ASCIIZ (null-terminated) string.

MAXEXEC2

Low-level MAX Database DLL. Support and utility routines for 32 bit applications.

MAXORDR2

32 bit DLL formerly known as MAXOrder.dll. Code for dealing with the various types of MAX Orders.

MAXTRAN2

MAX Transaction DLL. Code for processing MAX Inventory and Shop Floor Control Transactions.

OBJHANDLE

Defined as void far *. A 32-bit (segment:offset) pointer. This is the address of the Object Handle which must be used in calls to the DLLs.

.

16. Frequently Asked Questions

1. When I create a Sales Order using AddSalesOrderXML every field gets populated except ORDDTE_27.

- This field must be set in the XML which is passed into the AddSalesOrderXML function.

2. If I leave the ORDNUM_27 field blank, how does it determine what order number to use? Does it update the Configuration Data with the next available order number?

- If a blank order number is passed in the structure, the AddSalesOrderXML routine will use the next number from the Configuration Data. It also increments the next sales order number in the Configuration Data. If an order number is passed that already exists, the AddSalesOrderXML routine will continually increment the sales order number until it finds the next available sales order number to use.

3. If I let AddSalesOrderXML assign the order number, how do I retrieve that number?

- The return value of the AddSalesOrderXML function is a string which is the order number which was created. A blank string is returned in the case of an error.

4. If I let MAXUpdate generate the next order number is there a way to turn off or trap the message box that tells me the next number is being used?

- Yes, the AddSOXML function has a parameter to suppress all messages, it behaves just as AddSalesOrderXML in all other ways.

5. Where do you recommend installing the DLL's?

- We recommend completing an MAX client install on the workstation, verifying that this install works by logging into System Manager, and then using the DLLs in the EFW folder.

6. Does MAXUpdate allow processing Rework and/or Split functions?

- Split orders or rework orders have no API support in Windows.

7. What development tools can be used with MAXUpdate?

- Any development tool that allows the calling of functions within a DLL can be used with MAXUpdate. The most common tools are the .NET languages, Visual Basic and C++.

8. Do serial numbers need padding?

- Serial numbers do require right space padding also it is recommended to use right space padding for all strings when using MAXUpdate. Thus the serial number 1011 needs should be (using C# syntax) "1011".PadRight(30).

17. Common Problems

17.1 Application will not compile

- All MAX *.DLL components used in MAXUpdate must be located in the C:\Exact\RMClient\EFW folder.
- The development machine must have a path to the EFW folder.

17.2 Application does not run

- All MaxDlls must be considered as a set, MaxTran2.dll, MaxOrdr2.dll, MaxExec.dll, MaxSec.dll. They should be kept together
- The ERMRemCl.DLL and ExactRMEnc.DLL must be in the same directory as the application EXE file.
- Generally, your VB application must be in the EFW folder in order for your application to find MaxTrn2 or MaxOrdr2. Otherwise you will need a path statement for your application to find these DLLs.
- Failure to initialize MAXTransItem structure. Fields in this structure must be initialized before calling ProcessTrans. Failure to do so may result in un-initialized data being stored in reference fields. The resulting data will not be visible in some MAX screens, but will look normal when data is viewed directly in database. See ClearMAXTransItem in the Visual Basic sample for an example of how this works. This condition is due to the fact that Visual Basic does not initialize the elements of a structure.

17.3 Lot / Serial Control Problems

We have developed a sample application in which we use the functions from the Maxtran2.dll for a Purchase Order Receipt. See this application for examples.

17.4 Error Messages in Application

If your application runs but crashes in the same location, add the error handling routine from the sample code. Match the error code with the following table. For a large robust application, add the sample error handling code into your application and write a function to display the associated error test messages in the following table. Smaller applications don't need this additional work.

17.5 MAXUpdate Version Control

For consistent operation with MAX ERP, new MAX Update DLLs' are provided with MAX Service packs. Your custom application programs in production using one version of MAX Update DLLs' are generally compatible to new versions of MAXUpdate DLLs'. However it is good practice to test your specific custom application against a new release of MAXUpdate DLLs'.

17.6 Feedback

If you feel we have missed some important information please contact Exact Tech Support by entering a MAX Enhancement Request on the Exact Customer Portal.

18. MAXTrans Error Messages

If your custom program returns a negative error number add 65536 to the error number before matching numbers on the following table.

Resource ID	Value	Error Message String
IDS_NOMAXEXE	32777	Error: Can't find file MAX .EXE."
IDS_NOINPUTFILE	32778	Error: Need input File Name."
IDS_NOTRANSFILE	32779	No Transaction file specified"
IDS_BADTRANSTYPE	32780	Error: Illegal transaction type '%c'"
IDS_ERRORFCLOSE	32781	Error closing Input File."
IDS_ERRORFOPEN	32782	Can't open input file."
IDS_BADSUBTYPE	32783	Error: The subtype '%c' is not valid for the transaction type."
IDS_BADDATE	32784	Error: Transaction must have a valid date.."
IDS_BADTIME	32785	Error: Transaction must have a valid TIME of the form HHMMSS."
IDS_BADPRTNUM	32786	Error: Invalid Part Number - %s."
IDS_BADSTOCKID	32787	Error: Invalid Stockroom - %s."
IDS_BADSTOCKRESULT	32788	Error: On Adjustment resulting stock quantity must be >= 0.\n Part - %s, StkID - %s, Qty - %.04f"
IDS_CYLBADTNXQTY	32789	Error: On Cycle Counts the Transaction Quantity must be >= 0. Part - %s, StkID - %s"
IDS_OUTCOSTTOLERANCE	32790	Warning: The Cycle Count Quantity is out of $ tolerance. Part - %s"
IDS_OUTQTYTOLERANCE	32791	Warning: The Cycle Count Quantity is out of %% tolerance. Part - %s"
IDS_BADPARTSTOCKREL	32792	Error: Issuing stockroom has insufficient quantity to cover the transfer.\n Part - %s, StkID - %s"
IDS_NONEGATIVESHIP	32793	Error: Negative shipments are not allowed."
IDS_ORDERDOESNTEXIST	32794	Error: The order '%s' does not exist."
IDS_PARTNUMNOMATCH	32795	Error: Part number '%s' does not match order '%s'."
IDS_ORDERMUSTBECU	32796	Error: The order type must be CU. Order - %s"
IDS_BADORDERSTATUS	32797	Error: Order status must be 1, 2, or 3. Order - %s"
IDS_QTYTOOHIGH	32798	Error: Transaction Quantity must be less or equal than Due Quantity. Order - %s"
IDS_BADORDERTYPE	32799	Error: Order Type must be MS, MF, or SO. Order - %s, Type - %c"
IDS_BADISSUEORDERSTATUS	32800	Error: Order must have a status of 3 or 4. Order - %s"
IDS_QTYOUTTOLERANCE	32801	Warning: The Transaction Quantity is out of tolerance.\n Order - %s, Part - %s"
IDS_NOREQDET	32802	Error: The Requirement Detail record does not exist. Order - %s, Part - %s"

Resource ID	Value	Error Message String
IDS_NOJOBPRO	32803	Error: The Job Progress record does not exist. Order - %s, Oper - %s"
IDS_BADQTYREM	32804	Error: The Queue Quantity must be greater than 0 prior to this transaction.\n Order - %s, Oper - %s"
IDS_BADJOBQTY	32805	Error: The Transaction Quantity must be greater than 0 and less or equal than the Queue Quantity.\n Order - %s, Oper - %s"
IDS_NOSOESHIP	32806	Error: To ship a sales order, use the subtype 'O'.\n Order - %s"
IDS_BADORDMST	32807	Error: The ORDMST order does not exist. Order - %s"
IDS_NOVOTYPE	32808	Error: Type 'VO' orders are not currently supported.\n Order - %s"
IDS_BADPARTTYPE	32809	Error: Part Type cannot be 'F' or 'P' for PO Receipt.\n Order - %s, Part - %s, Type - %c"
IDS_TOOMUCHRCPT	32810	Error: Receipt Quantity exceeds allowable excess. Order - %s"
IDS_INSPECTIONREQD	32811	Warning: An inspection is required on this receipt. Order - %s"
IDS_BADPONOTE	32812	Error: The PO Note order does not exist. Order - %s"
IDS_NOGLREF	32813	Error: The GL Reference is required. Order - %s"
IDS_BADGLREF	32814	Error: The GL Reference '%s' does not exist."
IDS_TNXQTYLOW	32815	Error: The Transaction Quantity must be greater than 0. Part - %s, Type - %c"
IDS_COMPONENTSHORTAGE	32816	Warning: Shortage on issuing part %s for order %s."
IDS_BADPOTYPE	32817	Error: Type on a PO receipt must be 'PO'. Order - %s"
IDS_PARTUNDERLOTCONTROL	32818	Error: Part '%s' is under Lot/Serial Control."
IDS_BADORDERTYPE2	32819	Error: OrdMst type must be MS or MF. Order - %s"
IDS_BADORDERSTATUS2	32820	Error: OrdMst status must be < 5. Order - %s"
IDS_BADJOBPRO	32821	Error: The Job Progress record does not exist.\n Order - %s, Oper - %s"
IDS_BADJOBPROSTATUS	32822	Error: The JobPro status must be 3 or 4.\n Order - %s, Oper - %s"
IDS_BADSTARTDATE	32823	Error: The Time Ticket date precedes the current start date of the operation.\n Order - %s, CurrStart - %s"
IDS_BADINDIRECTLABORACCT	32824	Error: The Indirect Labor Acct '%s' does not exist."
IDS_NOWORKCENTER	32825	Error: No Work Center exists for the current operation.\n Order - %s, Oper - %s"

Resource ID	Value	Error Message String
IDS_NOSOEDET	32826	Error: The SOEDet record does not exist. Order - %s"
IDS_UNPROCDMSORDER	32827	Cannot ship because of an unprocessed MS order (Feature Option).\n Order - %s"
IDS_CUSTHOLD_SHIPWARNING	32828	Warning: The customer '%s' is on Credit Hold."
IDS_CANTSHIP_STOCK	32829	Cannot ship: Insufficient stock exists for Line Item.\n Order - %s, Stock - %s, Quantity - %.04f"
IDS_INVALIDSHIFT	32830	Error: The Shift must be between 1 and 3. Shift - %d"
IDS_COMPQTYTOOHIGH	32831	Error: The completion transaction quantity exceeds the remaining quantity.\n Order - %s, Oper - %s"
IDS_INVALIDRUNACT	32832	Error: The Actual Runtime must be > 0. Order - %s, Oper - %s"
IDS_INVALIDRUNTTL	32833	Error: The Total Runtime must be > 0. Order - %s, Oper - %s"
IDS_DUPSTARTTIME	32834	Error: Duplicate start time for same Date and Employee.\n Employee - %s, Order - %s, Date - %s, Time - %s"
IDS_TOOMANYCOMP	32835	Error: Quantity complete is greater than queue. Order - %s, Oper - %s"
IDS_INSUFFQUE	32836	Error: Insufficient queue for negative quantity. Order - %s, Oper - %s"
IDS_TOOMUCHSCRAP	32837	Error: Quantity scrapped is greater than queue - Complete. Order - %s, Oper - %s"
IDS_PARTTYPENOTISSUED	32838	Error: Part type F, R, T, X or Y not issued. Order - %s, PartType - %c"
IDS_INSUFFSTOCK	32839	Error: Insufficient stock available in the location. Part - %s, Stk - %s"
IDS_NOSTOCKAVAIL	32840	Error: No Stock available in the location. Part - %s, Stk - %s"
IDS_BADOPERTYPE	32841	Error: Invalid Operation Type Code. Order - %s, Oper - %s, OpType - %c"
IDS_OPERONHOLD	32842	Error: The operation is on hold. Order - %s, Oper - %s"
IDS_BADWRKCTR	32843	Error: The Work Center does not exist. Order - %s, Oper - %s"
IDS_BADOPERSEQ	32844	Error: The Rework operation must be earlier that the current operation.\n Order - %s, Oper - %s"
IDS_TNXQTY_MUST_BE_LE_QUEUE	32845	Error: The Transaction Quantity must be less than or equal to the Queue Quantity.\n Order - %s, Oper - %s\n"
IDS_QTYEXCEEDSNEXTQUE	32846	Error: Transaction Quantity is greater than Next Operation Queue. Order - %s, Oper - %s"
IDS_QTYEXCEEDSPOSTED	32847	Warning: Transaction Quantity is greater than Posted Completions. Order - %s, Oper - %s"

Resource ID	Value	Error Message String
IDS_QTYEXCEEDSQUE	32848	Error: Transaction Quantity is greater than queue. Order - %s, Oper - %s"
IDS_QTYEXCEEDSLOAD	32849	Error: Transaction Quantity is greater than load. Order - %s, Oper - %s"
IDS_NOPULLTHRU	32850	Error: Pull-Through not possible (insufficient qty in queue).\n Order - %s, Oper - %s"
IDS_NOSOE	32851	Error: SOE is not on. Can't ship Sales Order '%s'."
IDS_ACCTNGON	32852	Error: Accounting is on. Can't ship Customer Demand order '%s'."
IDS_NONEGQTY	32853	Error: The Transaction Quantity must be >= 0. Order - %s"
IDS_COMPONENTUNDERLOTCTRL	32854	Error: Component %s is under Lot/Serial Control"
IDS_ISSUELOGFORMAT	32855	%-14s %-15s %-15s %-10s %10s\n"
IDS_NOPHANTOM	32856	Error: Transaction not allowed on P (Phantom) type Part. Part Number - %s, Order - %s"
IDS_NOFAMILY	32857	Error: Transaction not allowed on F (Family) type Part. Part Number - %s, Order - %s"
IDS_NOREPETITIVE	32858	Error: Transaction not allowed when Repetitive option is off."
IDS_NOLOTHEADER	32859	Error: The Lot Header record does exist. Part - %s, Lot - %s"
IDS_NOSERIALMASTER	32860	Error: The Serial Master record does exist. Part - %s, Serial No - %s"
IDS_BADSERIALSTOCK	32861	Error: Part %s, Serial Number %s is already in stock"
IDS_NOLOTNUMBER	32862	Error: Part '%s' is under Lot Control and no Lot Number was entered"
IDS_NOSERIALNUMBER	32863	Error: Part %s, Serial Number %s not found"
IDS_NOPARTLOT	32865	Error: The Part Lot record does not exist. Part - %s Lot - %s"
IDS_INSUFFICIENTINV	32866	Error: Insufficient inventory Lot %s"
IDS_BLANKSERIAL	32867	Error: There was a blank serial number in the series"
IDS_NOLOTTRACKING	32868	Error: Lot Tracking is not enabled"
IDS_NOTENOUGHMEMORY	32869	Error: Not enough memory to process a transaction with individual serial numbers"
IDS_BADMULRECORDER	32870	Error: A quantity has already been received into this lot. A different lot number or a different delivery must be used when the Multi-Receipts flag is 'N'.
IDS_CREATEDSENMST_WARNING	32873	Warning: A new Serial Master record was created for Part %s, Serial %s"
IDS_CANNOT_POST_CUR_NEGCOMQTY	32874	Error: Can not post a negative completion quantity in the current operation. A completion quantity has already been posted in the next operation."

Resource ID	Value	Error Message String
IDS_CREATEDLOTHDR_WARNING	32880	Warning: A new Lot Header record was created for Part %s, Lot %s"
IDS_LOTMISMATCH	32882	Error: Lot %s does not match the lot number in Order %s"
IDS_NOCOMPONENTS	32883	Error: No stock to issue part %s for order %s."
IDS_BADREQDETSTATUS	32884	Error: Requirement Detail must have a status of Open. Part %s, Order %s"
IDS_FUTURE_SHIP_DATE	32885	Error: The actual ship date cannot be a future date - Date %s."
IDS_BAD_PAST_SHIP_DATE	32886	Error: The actual ship date cannot be more than 45 days in the past - Date %s."
IDS_BADREQQTY	32888	Error: The scrap quantity can not be greater than the quantity issued.\n Order %s, Part %s."
IDS_BAD_NEG_SCRAP	32889	Error: The negative scrap amount can not be greater than the quantity already scrapped. Order %s, Part %s."
IDS_BAD_COMP_SCRAP_ORDER_STATUS	32890	Error: Order must have a status of 3. Order - %s."
IDS_NEG_ISSUE_TOO_HIGH	32891	Error: The amount of a negative issue can not be greater than the amount previously issued. Order %s, Part %s"
IDS_LOW_STOCK	32892	Warning: Issuing stockroom has insufficient quantity to cover the issue.\nAny available inventory was used. Part - %s, StkID - %s"
IDS_SHIP_WITH_SOE	32893	Error: Shipment transaction cannot used with Sales Order processing, use the subtype 'O'.\n Order - %s"
IDS_TNFBADTNXQTY	32894	Error: On Transfer the Transaction Quantity must be >= 0. Part - %s, StkID - %s"
IDS_QTYEXCEEDS_ORDERQTY	33025	Error: The negative receipt amount for order %s exceeds the amount previously received."
IDS_INVALID_DEFECT_CODE	33026	Error: %s is an invalid defect code."
IDS_NO_DEFECT_CODE	33027	Error: A defect code is required. Order - %s."
IDS_SCRAP_QTY_GT_VENDOR_QTY	33028	Error: The scrap quantity exceeds the quantity at the vendor. Order - %s."
IDS_NEGATIVE_SCRAP_EXCEEDS_SCRAP_QTY	33029	Error: The quantity being unscrapped exceeds the quantity previously scrapped. Order - %s."
IDS_ORDER_NOT_SERVICE_ORDER	33030	Error: The order must be a Service Order. Order - %s."
IDS_INVALID_EMPLOYEE	33032	Error: %s is not a valid Employee ID."
IDS_SINGLE_PRIVILEGE	33033	Error: Employee,%s, has single privilege and can only have one work order open at a time"

Resource ID	Value	Error Message String
IDS_INVALID_LOGOUT_DATE	33034	Error: The Logout date is earlier than the Login date.\n Employee - %s, Order - %s, Date - %s, Time - %s.
IDS_INVALID_LOGOUT_TIME	33035	Error: The Logout time is earlier than the Login time.\n Employee - %s, Order - %s, Date - %s, Time - %s.
IDS_INVALID_INDIRECT_LOGOUT_DATE	33036	Error: The Logout date is earlier than the Login date.\n Employee - %s, Date - %s, Time - %s."
IDS_INVALID_INDIRECT_LOGOUT_TIME	33037	Error: The Logout time is earlier than the Login time.\n Employee - %s, Date - %s, Time - %s
IDS_ISSUE_TOO_HIGH	59142	Warning: The issue quantity is greater than the requirement due quantity - Order %s, Part %s."
IDS_RECEIVING_CLOSED_ORDER	59143	Warning: A closed line item is being received for order %s."
IDS_NO_ASSIGNMENT_CODE	59144	Error: There is no Serial Assignment Code for Serial - %s."
IDS_BAD_NEG_ISSUE_LOT	59145	Error: The lot used for a negative issue must be an existing lot. Order %s, Part %s."
IDS_ERROR_GTRANSOPEN	59149	No transaction file specified."
IDS_INVALID_FILE	59150	Error: The input file is invalid or corrupted."
IDS_STRANSFILE_ERROR	59151	Error: Can't update temporary transaction file."
IDS_NOTENOUGHMEMORY_LOT	59152	Error: Not enough memory to process a cycle count transaction with lot numbers."
IDS_INSUFFICIENTSERIALS	59153	Error: Not enough serial numbers."
IDS_INSUFFICIENT_LOT_QTY	59154	Error: The individual lot quantities do not add up to the cycle count transaction quantity - Part %s, Stk %s."
IDS_UNSHIPQTYTOOHIGH	59155	Error: The return quantity cannot exceed the quantity previously shipped.\n Order - %s."
IDS_MULTI_RECEIPTS_AND_LOTSFC	59156	Error: Partial receipts are not allowed when the Multi-Receipts flag is set to ""No"" and the Lot SFC flag is set to ""Yes"". "
IDS_OUTSIDE_WORKCENTER_AND_PULLTHRU	59157	Error: Can not pull a quantity through a subcontract operation.
IDS_NO_STOCKROOM	59158	Error: Part %s does not have a stock room defined. It will not be received into stock."
IDS_SERIAL_NOT_ISSUED	59159	Error: A sufficient quantity has not been issued to serial number %s for order %s."
IDS_LOT_NOT_ISSUED	59160	Error: A sufficient quantity has not been issued to lot number %s for order %s."
IDS_LOT_SERIAL_NOT_ISSUED	59161	Error: A sufficient quantity has not been issued to Lot/Serial %s for order %s."

Resource ID	Value	Error Message String
IDS_SCRAP_NOT_ALLOWED	59162	Error: The scrap allowed flag in Labor Tracking is set to No and scrap is non-zero. Order - %s, Opr - %s."
IDS_NEG_SETUP_TIME	59163	Error: The setup time must greater than zero. Order - %s, Opr - %s."
IDS_NEG_RUNTIME	59164	Error: The run time value must be greater than zero. Order - %s, Opr - %s."
IDS_INCORRECT_LOGOUT	59165	Error: The Logout date and time can not be before the Login date and time. Order - %s, Opr - %s."
IDS_TIME_GT_ELAPSED	59167	Error: The Setup time plus the Runtime can not be greater than the elapsed time. Order - %s, Opr - %s."
IDS_INVALID_INDIRECT_ACCOUNT_CODE	59168	Error: %s is not a valid Indirect Account code
IDS_ALREADY_LOGGED_IN	59169	There is already an open Indirect work record. You can only have one open indirect work record per employee.
IDS_NO_MATCHING_EMPWRK	59170	There are no matching Employee Work records for Employee - %s, Order - %s and Operation Sequence - %s.
IDS_NO_ACTIVE_INDIRECT_EMPWRK	59171	There are no active Employee Work records for Employee - %s.
IDS_ERR_TRANSFER_UNDO_QTY	59172	Error: Undo transfer to vendor before posting negative.

19. Working XML of Transactions

19.1 Receipt - Non-Inventory

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>N</SUBTYPE_39>
			<ORDNUM_39>70000021</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<TNXQTY_39>5</TNXQTY_39>
			<GLREF_39>MSC</GLREF_39>
			<REFDSC_39>Test</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.2 Receipt – Purchase

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>P</SUBTYPE_39>
			<ORDNUM_39>70000022</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>01</DELNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<RCVSTK_39>FGI</RCVSTK_39>
			<REFDSC_39>Test</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.3 Receipt – RMA

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>U</SUBTYPE_39>
			<PRTNUM_39>13100</PRTNUM_39>
			<ORDNUM_39>80000005</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
			<RCVSTK_39>FGI</RCVSTK_39>
			<REFDSC_39></REFDSC_39>
			<GLREF_39>MSC</GLREF_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.4 Receipt – Shop

```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>S</SUBTYPE_39>
			<ORDNUM_39>50000009</ORDNUM_39>
			<LINNUM_39>00</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<RCVSTK_39>FGI</RCVSTK_39>
			<REFDSC_39>dfdsf</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.5 Receipt – Subcontract

```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>O</SUBTYPE_39>
			<ORDNUM_39>70000027</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>01</DELNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<RCVSTK_39>RMI</RCVSTK_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.6 Receipt – Unplanned

```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>U</SUBTYPE_39>
			<PRTNUM_39>13100</PRTNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<RCVSTK_39>RMI</RCVSTK_39>
			<GLREF_39>MSC</GLREF_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.7 Issue – Shop

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>I</TYPE_39>
			<SUBTYPE_39>S</SUBTYPE_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<PRTNUM_39>13100</PRTNUM_39>
			<ORDNUM_39>50000021</ORDNUM_39>
			<LINNUM_39>00</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.8 Issue – Subcontract

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>I</TYPE_39>
			<SUBTYPE_39>O</SUBTYPE_39>
			<ORDNUM_39>70000030</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>01</DELNUM_39>
			<PRTNUM_39>13105</PRTNUM_39>
			<TNXQTY_39>4</TNXQTY_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<REFDSC_39>Test</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.9 Issue – Unplanned

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>I</TYPE_39>
			<SUBTYPE_39>U</SUBTYPE_39>
			<PRTNUM_39>11000</PRTNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<REFDSC_39>Test</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.10 Shipment - Sales Order


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>S</TYPE_39>
			<SUBTYPE_39>M</SUBTYPE_39>
			<ORDNUM_39>20000018</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>01</DELNUM_39>
			<PRTNUM_39>11000</PRTNUM_39>
			<TNXQTY_39>2</TNXQTY_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<REFDSC_39>PowerServe 2299</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```


19.11 Shipment - Subcontract BOM


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>S</TYPE_39>
			<SUBTYPE_39>M</SUBTYPE_39>
			<ORDNUM_39>70000013</ORDNUM_39>
			<LINNUM_39>01</LINNUM_39>
			<DELNUM_39>01</DELNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```


19.12 Transfer


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>F</TYPE_39>
			<SUBTYPE_39></SUBTYPE_39>
			<PRTNUM_39>11000</PRTNUM_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<RCVSTK_39>ADS</RCVSTK_39>
			<TNXQTY_39>1</TNXQTY_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.13 Adjustment

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>A</TYPE_39>
			<SUBTYPE_39></SUBTYPE_39>
			<PRTNUM_39>11000</PRTNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
			<ISSSTK_39>FGI</ISSSTK_39>
			<GLREF_39>MSC</GLREF_39>
			<REFDSC_39>ROHIT REF</REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.14 Cycle-Count

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>C</TYPE_39>
			<SUBTYPE_39> </SUBTYPE_39>
			<ISSSTK_39>RMIOVER</ISSSTK_39>
			<PRTNUM_39>11000</PRTNUM_39>
			<TNXQTY_39>20</TNXQTY_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.15 Repetitive

```
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>B</TYPE_39>
			<SUBTYPE_39> </SUBTYPE_39>
			<PRTNUM_39>13100</PRTNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
			<RCVSTK_39>FGI</RCVSTK_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.16 Receipt – Backflush


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>R</TYPE_39>
			<SUBTYPE_39>B</SUBTYPE_39>
			<ORDNUM_39>50000007</ORDNUM_39>
			<LINNUM_39>00</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<TNXQTY_39>1</TNXQTY_39>
			<RCVSTK_39>FGI</RCVSTK_39>
			<REFDSC_39></REFDSC_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```


19.17 Labor - Login Indirect


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>L</TYPE_39>
			<SUBTYPE_39>I</SUBTYPE_39>
			<TNXQTY_39>1</TNXQTY_39>
			<TNXDTE_39>130716</TNXDTE_39>
			<TNXTME_39>020447</TNXTME_39>
			<SHIFT_39>1</SHIFT_39>
			<EMPID_39>1200</EMPID_39>
			<GLREF_39>A</GLREF_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```


19.18 Labor - Login Order


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>L</TYPE_39>
			<SUBTYPE_39>L</SUBTYPE_39>
			<ORDNUM_39>50000012</ORDNUM_39>
			<LINNUM_39>00</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<OPRSEQ_39>0040</OPRSEQ_39>
			<TNXQTY_39>1</TNXQTY_39>
			<TNXDTE_39>130716</TNXDTE_39>
			<TNXTME_39>020517</TNXTME_39>
			<SHIFT_39>1</SHIFT_39>
			<EMPID_39>1200</EMPID_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.19 Labor - Logout Indirect


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>L</TYPE_39>
			<SUBTYPE_39>D</SUBTYPE_39>
			<TNXQTY_39>1</TNXQTY_39>
			<TNXDTE_39>130716</TNXDTE_39>
			<TNXTME_39>020543</TNXTME_39>
			<EMPID_39>1200</EMPID_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```


19.20 Labor - Logout Order


```xml
<?xml version="1.0" encoding="utf-8" ?>
<eMAXExact>
	<MAX_Transaction_Table>
		<MAX_Transaction>
			<TYPE_39>L</TYPE_39>
			<SUBTYPE_39>O</SUBTYPE_39>
			<ORDNUM_39>50000012</ORDNUM_39>
			<LINNUM_39>00</LINNUM_39>
			<DELNUM_39>00</DELNUM_39>
			<OPRSEQ_39>0040</OPRSEQ_39>
			<TNXDTE_39>130716</TNXDTE_39>
			<TNXTME_39>020616</TNXTME_39>
			<TNXQTY_39>1</TNXQTY_39>
			<SETUPTIME_39>0</SETUPTIME_39>
			<SHIFT_39>1</SHIFT_39>
			<EMPID_39>1200</EMPID_39>
			<ASCRAP_39>0</ASCRAP_39>
			<EXPDATE_39>130523</EXPDATE_39>
			<TICKET_39>011750</TICKET_39>
		</MAX_Transaction>
	</MAX_Transaction_Table>
</eMAXExact>
```

19.21 AddSalesOrderHeader


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<SO_Master_Table>
		<SO_Master>
			<CUSTID_27>300</CUSTID_27>
			<GLXREF_27>000410000</GLXREF_27>
			<STYPE_27>CU</STYPE_27>
			<STATUS_27>3</STATUS_27>
			<ORDDTE_27>2013-07-16</ORDDTE_27>
			<REP1_27>MW01</REP1_27>
			<SPLIT1_27>100</SPLIT1_27>
			<TERMS_27>03</TERMS_27>
			<SHIPVIA_27 />
			<FOB_27>Shipping Point</FOB_27>
			<TAXCD1_27>CHICITY</TAXCD1_27>
			<TAXCD2_27>COKCNTY</TAXCD2_27>
			<TAXCD3_27>ILSTATE</TAXCD3_27>
			<INVCE_27>N</INVCE_27>
			<NAME_27>Saturn Automotive</NAME_27>
			<ADDR1_27>3400 Lake Shore Drive</ADDR1_27>
			<CITY_27>Chicago</CITY_27>
			<STATE_27>IL</STATE_27>
			<ZIPCD_27>60615</ZIPCD_27>
			<CNTRY_27>US</CNTRY_27>
			<PHONE_27>312-982-2000</PHONE_27>
			<CNTCT_27>Tim O'Leary</CNTCT_27>
			<SHPCDE_27 />
			<SHPVIA_27>01</SHPVIA_27>
			<FEDTAX_27>N</FEDTAX_27>
			<TAXABL_27>Y</TAXABL_27>
			<EXCRTE_27>1</EXCRTE_27>
			<FIXVAR_27>F</FIXVAR_27>
			<CURR_27>US</CURR_27>
			<TTAX_27>0</TTAX_27>
			<LNETAX_27>N</LNETAX_27>
		</SO_Master>
	</SO_Master_Table>
</eMAXExact>
```

19.22 ChangeSalesOrderHeader


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<SO_Master_Table>
		<SO_Master>
			<ORDNUM_27>20000071</ORDNUM_27>
			<CUSTID_27>300</CUSTID_27>
			<GLXREF_27>000410000</GLXREF_27>
			<STYPE_27>CU</STYPE_27>
			<STATUS_27>3</STATUS_27>
			<ORDDTE_27>2013-07-16</ORDDTE_27>
			<REP1_27>MW01</REP1_27>
			<SPLIT1_27>100</SPLIT1_27>
			<TERMS_27>03</TERMS_27>
			<SHIPVIA_27 />
			<FOB_27>Shipping Point</FOB_27>
			<TAXCD1_27>CHICITY</TAXCD1_27>
			<TAXCD2_27>COKCNTY</TAXCD2_27>
			<TAXCD3_27>ILSTATE</TAXCD3_27>
			<INVCE_27>N</INVCE_27>
			<NAME_27>Saturn Automotive</NAME_27>
			<ADDR1_27>3400 Lake Shore Drive</ADDR1_27>
			<CITY_27>Chicago</CITY_27>
			<STATE_27>IL</STATE_27>
			<ZIPCD_27>60615</ZIPCD_27>
			<CNTRY_27>US</CNTRY_27>
			<PHONE_27>312-982-2000</PHONE_27>
			<CNTCT_27>Tim O'Leary</CNTCT_27>
			<SHPCDE_27 />
			<SHPVIA_27>01</SHPVIA_27>
			<FEDTAX_27>N</FEDTAX_27>
			<TAXABL_27>Y</TAXABL_27>
			<EXCRTE_27>1</EXCRTE_27>
			<FIXVAR_27>F</FIXVAR_27>
			<CURR_27>US</CURR_27>
			<TTAX_27>0</TTAX_27>
			<LNETAX_27>N</LNETAX_27>
		</SO_Master>
	</SO_Master_Table>
</eMAXExact>
```


19.23 DeleteSalesOrderXML(ordernum)

19.24 AddSalesOrderLine


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<SO_Detail_Table>
		<SO_Detail>
			<ORDNUM_28>20000070</ORDNUM_28>
			<LINNUM_28>03</LINNUM_28>
			<DELNUM_28>01</DELNUM_28>
			<STATUS_28>3</STATUS_28>
			<CUSTID_28>200</CUSTID_28>
			<PRTNUM_28>11000</PRTNUM_28>
			<TAXABL_28>Y</TAXABL_28>
			<GLXREF_28>000451001</GLXREF_28>
			<CURDUE_28>2013-07-30T00:00:00</CURDUE_28>
			<ORGDUE_28>2013-07-30T00:00:00</ORGDUE_28>
			<CUSDUE_28>2013-07-30T00:00:00</CUSDUE_28>
			<SLSUOM_28>EA</SLSUOM_28>
			<ORGPRC_28>1099</ORGPRC_28>
			<PRICE_28>1099</PRICE_28>
			<ORGQTY_28>1</ORGQTY_28>
			<CURQTY_28>1</CURQTY_28>
			<DUEQTY_28>1</DUEQTY_28>
			<DISC_28>0</DISC_28>
			<STYPE_28>CU</STYPE_28>
			<PRNT_28>N</PRNT_28>
			<AKPRNT_28>N</AKPRNT_28>
			<STK_28>FGI</STK_28>
			<FORCUR_28>1099</FORCUR_28>
			<HSTAT_28>R</HSTAT_28>
			<TAXCDE1_28>BOSCITY</TAXCDE1_28>
			<TAXCDE2_28>MIDCNTY</TAXCDE2_28>
			<TAXCDE3_28>MASTATE</TAXCDE3_28>
			<TAX1_28>3297</TAX1_28>
			<TAX2_28>3297</TAX2_28>
			<TAX3_28>4396</TAX3_28>
		</SO_Detail>
	</SO_Detail_Table>
</eMAXExact>
```

19.25 ChangeSalesOrderLine

Old Xml


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<SO_Detail_Table>
		<SO_Detail>
			<ORDNUM_28>20000070</ORDNUM_28>
			<LINNUM_28>03</LINNUM_28>
			<DELNUM_28>01</DELNUM_28>
			<STATUS_28>3</STATUS_28>
			<CUSTID_28>200                 </CUSTID_28>
			<PRTNUM_28>11000                          </PRTNUM_28>
			<TAXABL_28>Y</TAXABL_28>
			<GLXREF_28>000451001                        </GLXREF_28>
			<CURDUE_28>7/30/2013 12:00:00 AM</CURDUE_28>
			<ORGDUE_28>7/30/2013 12:00:00 AM</ORGDUE_28>
			<CUSDUE_28>7/30/2013 12:00:00 AM</CUSDUE_28>
			<SLSUOM_28>EA</SLSUOM_28>
			<ORGPRC_28>1099</ORGPRC_28>
			<PRICE_28>1099</PRICE_28>
			<ORGQTY_28>1</ORGQTY_28>
			<CURQTY_28>1</CURQTY_28>
			<DUEQTY_28>1</DUEQTY_28>
			<DISC_28>0</DISC_28>
			<STYPE_28>CU</STYPE_28>
			<PRNT_28>N</PRNT_28>
			<AKPRNT_28>N</AKPRNT_28>
			<STK_28>FGI</STK_28>
			<FORCUR_28>1099</FORCUR_28>
			<HSTAT_28>R</HSTAT_28>
			<TAXCDE1_28>BOSCITY</TAXCDE1_28>
			<TAXCDE2_28>MIDCNTY</TAXCDE2_28>
			<TAXCDE3_28>MASTATE</TAXCDE3_28>
			<TAX1_28>3297</TAX1_28>
			<TAX2_28>3297</TAX2_28>
			<TAX3_28>4396</TAX3_28>
		</SO_Detail>
	</SO_Detail_Table>
</eMAXExact>
```


New XML


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<SO_Detail_Table>
		<SO_Detail>
			<ORDNUM_28>20000070</ORDNUM_28>
			<LINNUM_28>03</LINNUM_28>
			<DELNUM_28>01</DELNUM_28>
			<STATUS_28>3</STATUS_28>
			<CUSTID_28>200                 </CUSTID_28>
			<PRTNUM_28>11000                          </PRTNUM_28>
			<TAXABL_28>Y</TAXABL_28>
			<GLXREF_28>000451001</GLXREF_28>
			<CURDUE_28>2013-07-30T00:00:00</CURDUE_28>
			<ORGDUE_28>2013-07-30T00:00:00</ORGDUE_28>
			<CUSDUE_28>2013-07-30T00:00:00</CUSDUE_28>
			<SLSUOM_28>EA</SLSUOM_28>
```

```
                    <ORGPRC_28>1099</ORGPRC_28>
                    <PRICE_28>1099</PRICE_28>
                    <ORGQTY_28>2</ORGQTY_28>
                    <CURQTY_28>2</CURQTY_28>
                    <DUEQTY_28>2</DUEQTY_28>
                    <DISC_28>0</DISC_28>
                    <STYPE_28>CU</STYPE_28>
                    <PRNT_28>N</PRNT_28>
                    <AKPRNT_28>N</AKPRNT_28>
                    <STK_28>FGI</STK_28>
                    <FORCUR_28>2198</FORCUR_28>
                    <HSTAT_28>R</HSTAT_28>
                    <TAXCDE1_28>BOSCITY</TAXCDE1_28>
                    <TAXCDE2_28>MIDCNTY</TAXCDE2_28>
                    <TAXCDE3_28>MASTATE</TAXCDE3_28>
                    <TAX1_28>6594</TAX1_28>
                    <TAX2_28>6594</TAX2_28>
                    <TAX3_28>8792</TAX3_28>
                </SO_Detail>
            </SO_Detail_Table>
</eMAXExact>
```


19.26 DeleteSalesOrderLineItemXML(ordernum)

19.27 ConvertQuoteXML(ordernum, ref bSuppressMesaage)

19.28 AddRMA


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<RMA_Detail_Table>
		<RMA_Detail>
			<RMANUM_52>80000009</RMANUM_52>
			<RMALIN_52>01</RMALIN_52>
			<RMADEL_52>00</RMADEL_52>
			<RMADTE_52>20130715</RMADTE_52>
			<ORDNUM_52>20000010</ORDNUM_52>
			<LINNUM_52>01</LINNUM_52>
			<DELNUM_52>01</DELNUM_52>
			<POS_52>0000</POS_52>
			<CUSTID_52>100</CUSTID_52>
			<RETNUM_52>11000</RETNUM_52>
			<RETQTY_52>0</RETQTY_52>
			<RETDAY_52>0</RETDAY_52>
			<LOTNUM_52 />
			<SERIAL_52 />
			<REPAIR_52>N</REPAIR_52>
			<RPLACE_52>N</RPLACE_52>
			<CREDIT_52>N</CREDIT_52>
			<STK_52 />
			<RESPON_52 />
			<NEWORD_52 />
			<NEWLIN_52 />
			<NEWDEL_52 />
			<WRKNUM_52 />
			<KEYWRD_52 />
			<RECQTY_52>0</RECQTY_52>
			<RECDTE_52 />
			<DUEQTY_52>0</DUEQTY_52>
			<MRBDSP_52 />
			<STATUS_52>1</STATUS_52>
			<REM1_52 />
			<REM2_52 />
			<REM3_52 />
			<REM4_52 />
			<CMPDTE_52 />
			<WARNUM_52 />
			<MCOMP_52 />
			<MSITE_52 />
			<UDFKEY_52 />
			<UDFREF_52 />
		</RMA_Detail>
	</RMA_Detail_Table>
</eMAXExact>
```

19.29 ChangeRMA


```xml
<?xml version="1.0" encoding="utf-16"?>
<eMAXExact>
	<RMA_Detail_Table>
		<RMA_Detail>
			<RMANUM_52>80000007</RMANUM_52>
			<RMALIN_52>01</RMALIN_52>
			<RMADEL_52>00</RMADEL_52>
			<RMADTE_52>20130715</RMADTE_52>
			<ORDNUM_52 />
			<LINNUM_52 />
			<DELNUM_52 />
			<POS_52 />
			<CUSTID_52>100</CUSTID_52>
			<RETNUM_52>11000</RETNUM_52>
			<RETQTY_52>1</RETQTY_52>
			<RETDAY_52>1</RETDAY_52>
			<LOTNUM_52 />
			<SERIAL_52 />
			<REPAIR_52>N</REPAIR_52>
			<RPLACE_52>N</RPLACE_52>
			<CREDIT_52>N</CREDIT_52>
			<STK_52 />
			<RESPON_52 />
			<NEWORD_52 />
			<NEWLIN_52 />
			<NEWDEL_52 />
			<WRKNUM_52 />
			<KEYWRD_52 />
			<RECQTY_52>1</RECQTY_52>
			<RECDTE_52 />
			<DUEQTY_52>1</DUEQTY_52>
			<MRBDSP_52 />
			<STATUS_52>1</STATUS_52>
			<REM1_52 />
			<REM2_52 />
			<REM3_52 />
			<REM4_52 />
			<CMPDTE_52 />
			<WARNUM_52 />
			<MCOMP_52 />
			<MSITE_52 />
			<UDFKEY_52 />
			<UDFREF_52 />
		</RMA_Detail>
	</RMA_Detail_Table>
</eMAXExact>
```


19.30 DeleteRMAXML(RMANUM, RMALIN, RMADEL)